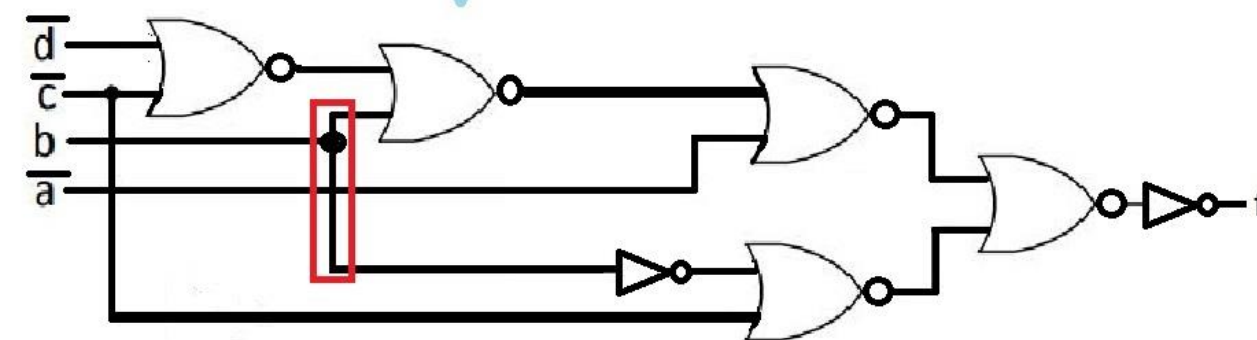
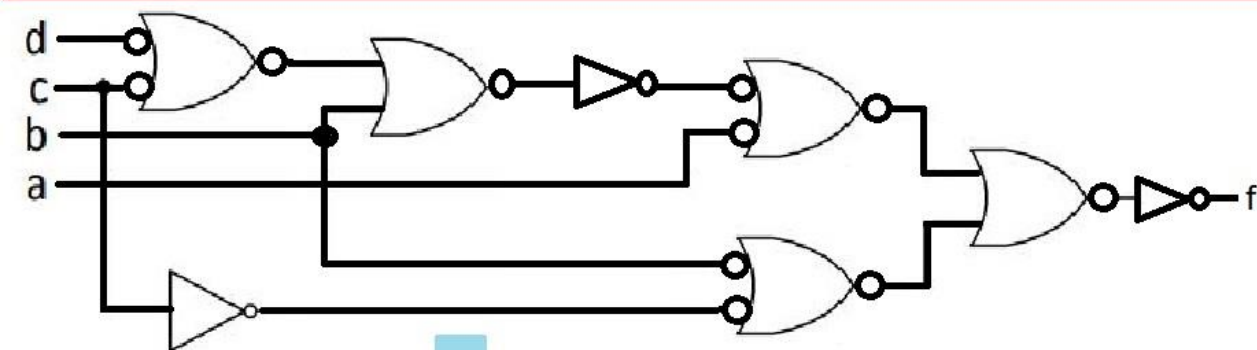
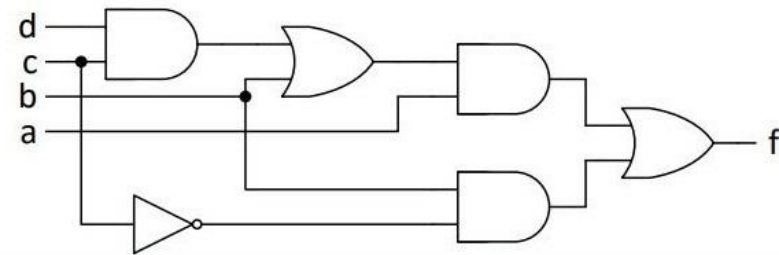


Zadatak: Logičku funkciju $f = a(b + cd) + b\bar{c}$ realizovati u obliku homogene kombinacione mreže dvoulaznih NILI kola.



Fokusirajmo se na ovu sliku za pocetak. Ona je sustinski bitna ovde. Ova ispod je samo doradjivanje ove iznad. Dakle ovako: Trazi se da se realizuje KM preko dvoulaznih NILI kola. Evo kako se to radi: znaci ako imamo I kolo, onda stavimo NILI sa invertovanim ulazima, a ako imamo ILI kolo, onda stavimo kruzic na kraju, pa onda jos i invertor. Naravno, sve ovo radimo pod uslovom da su kola dvoulazna, a ako ima neko koje nije dvoulazno nego viseulazno, onda se prvo to resava, pa tek onda gorenavedeni postupak. Tako da eto, to je to za ovaj zadatak.

Zadatak: Logičke funkcije $f(a, b, c)$ i $g(a, b, c)$ definisane Karnoovima mapama na slici realizovati pomoću PLA minimalnih dimenzija.

		f			
		bc 00	bc 01	bc 11	bc 10
a	0	1			
	1	1	1	1	

		g			
		bc 00	bc 01	bc 11	bc 10
a	0	1			
	1		1	1	1

		f			
		bc 00	bc 01	bc 11	bc 10
a	0	1			
	1	1	1	1	

Sta je ovde poenta? Pa poenta je u tome da treba da nadujemo jedinice na zajednickim pozicijama u ovim Karnoovim mapama. Dakle ne radimo klasicnu minimizaciju, vec zajednicke keceve trazimo. Kada to radimo, grupisemo ih. E sad, vrlo je vazno reci da ako imamo par jedinica zadatih na takav nacin, ako jedan ne grupisemo, onda ne mozemo ni ovaj drugi. Recimo, za g funkciju, za $abc = 000$ imamo keca na tom mestu kojeg ne mozemo da grupisemo, vec njega samog zaokruzujemo. Ali u funkciji f imamo keca na tom istom mestu, ali i na $abc = 100$ sa kojim bi mogli da ga zaokruzimo zajedno, s tim sto to necemo uraditi, jer bi to onda bilo netacno. Ovde je zadato kako se tacno resava ovaj zadatak, a dole cemo dati netacno resenje, kako bi se uocila razlika.

Jedinice koje nisu na zajednickim pozicijama uparujemo klasicno kako smo navikli da radimo minimizaciju Karnoovim mapama, a one koje jesu, e za njih bi vec trebalo da povedemo racuna sta sa njima radimo. Poenta zasto ovako radimo je da bi imali sto manji broj produktnih clanova, to je poenta.

		g			
		bc 00	bc 01	bc 11	bc 10
a	0	1			
	1		1	1	1



Dakle ovo je full tacno resenje, a u nastavku je dato kako NE treba raditi ovaj zadatak. Takodje, dato je i drugo tacno resenje. U ovom konkretnom slucaju, ne postoji samo jedno tacno resenje. Eto zato je to.

		f			
		bc 00	bc 01	bc 11	bc 10
a	0	1			
	1	1	1	1	



NETACNO !!!!

		g			
		bc 00	bc 01	bc 11	bc 10
a	0	1			
	1		1	1	1

		f			
		bc 00	bc 01	bc 11	bc 10
a	0	1			
	1	1	1	1	



NETACNO !!!!

		g			
		bc 00	bc 01	bc 11	bc 10
a	0	1			
	1		1	1	1

		f			
		bc 00	bc 01	bc 11	bc 10
a	0	1			
	1	1	1	1	

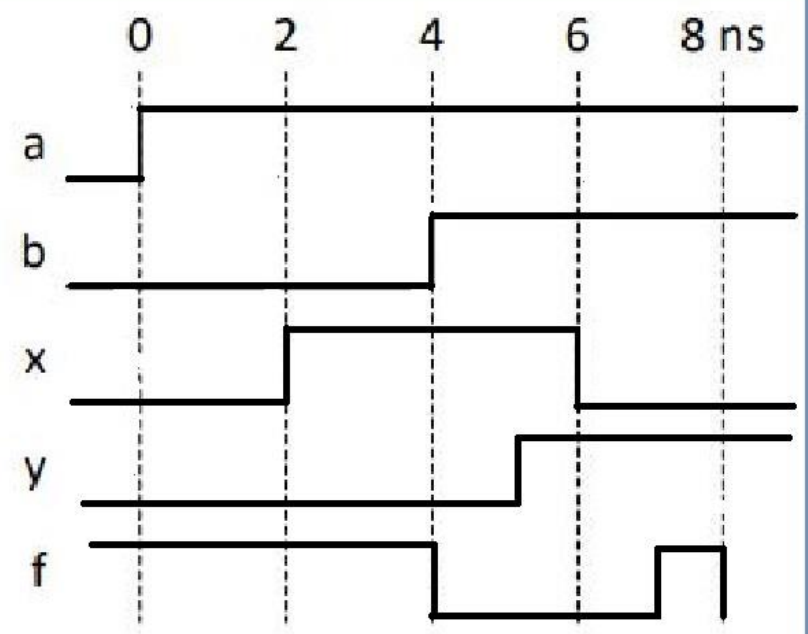
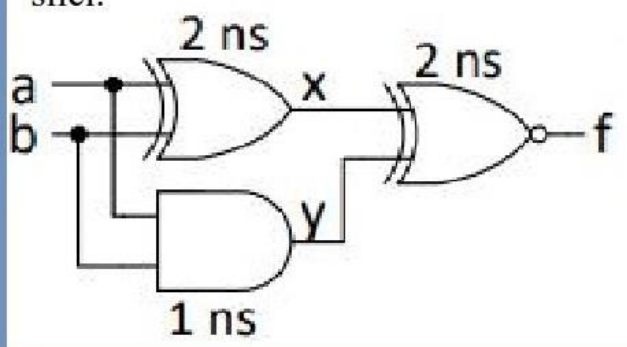


TACNO !!!!

UPOREDITI DVA TACNA RESENJA !!!!!!!

		g			
		bc 00	bc 01	bc 11	bc 10
a	0	1			
	1		1	1	1

Zadatak. Nacrtati talasne oblike signala na izlazu f i unutrašnjim tačkama x i y kombinacione mreže sa slike $i \rightarrow$, pod pretpostavkom da se ulazni signali, a i b , menjaju kao na slici $ii \rightarrow$. Propagaciona kašnjenja logičkih kola su naznačena na slici.



I kolo tabela istinitosti

a	b	y
0	0	0
0	1	0
1	0	0
1	1	1

XOR kolo tabela istinitosti

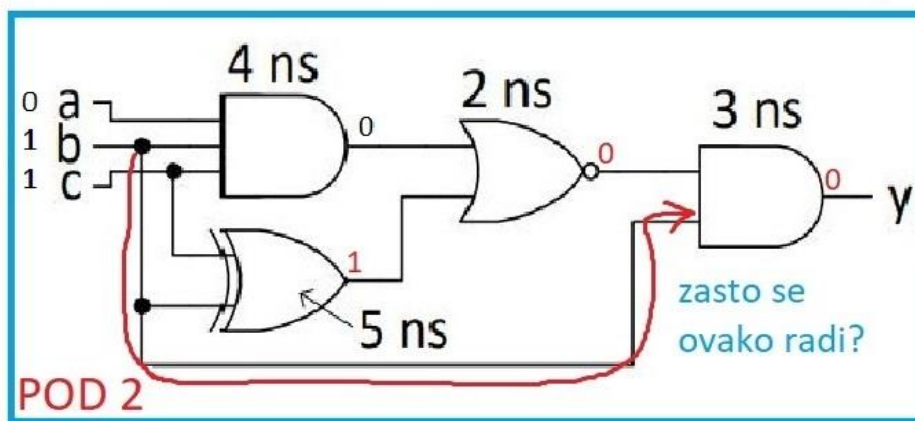
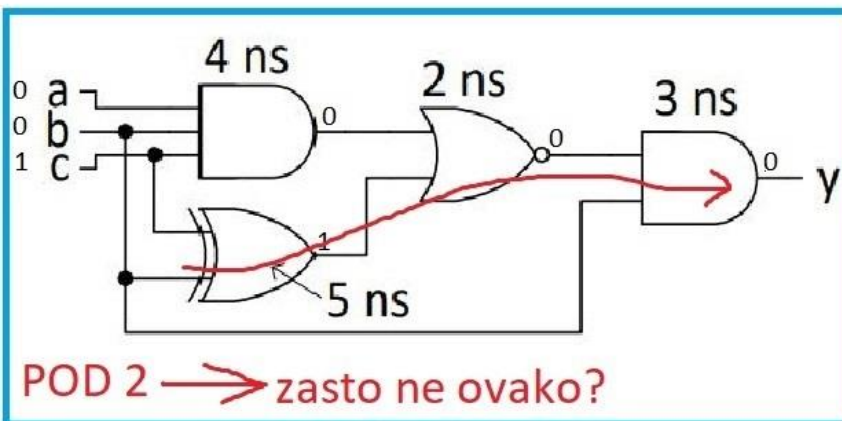
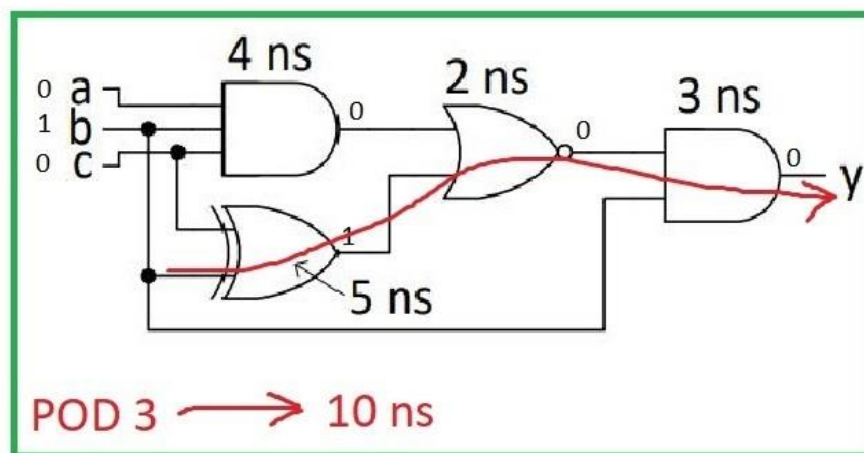
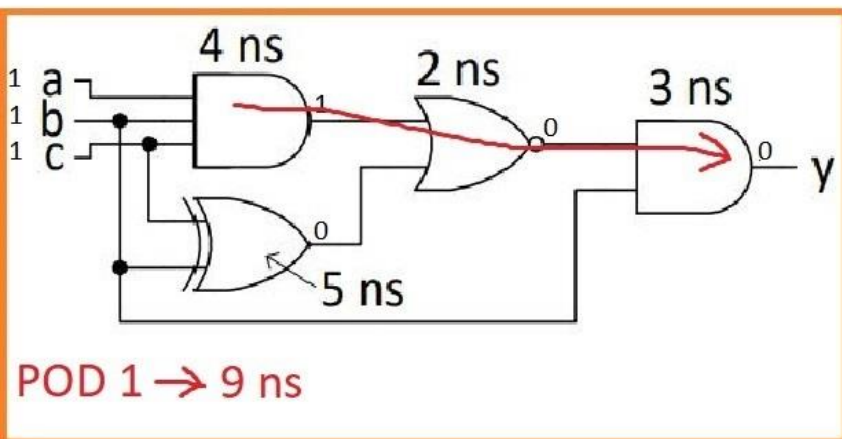
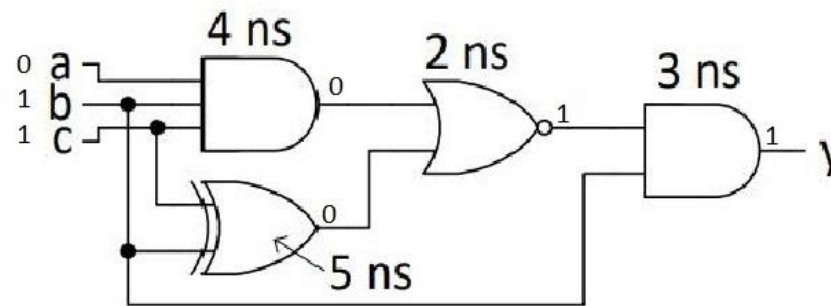
a	b	x
0	0	0
0	1	1
1	0	1
1	1	0

XNOR kolo tabela istinitosti

x	y	f
0	0	1
0	1	0
1	0	0
1	1	1

Ovde je samo poenta da na osnovu tabele istinitosti nacrtamo talasne oblike signala, s tim sto su oni pomereni za onoliko nanosekundi koliko ima kolo koje u tom trenutku proucavamo. Prost primer: od nulte do druge nanosekunde je $a = 1$ i $b = 0$, sto znaci da ce od druge do cetvrte nanosekunde x biti 1, jer je propagaciono kasnjenje XOR kola 2 nanosekunde. Zatim je, recimo, od 2. do 5. nanosekunde $x = 1$, kao i $y = 0$, pa je onda od 4. do 7. nanosekunde $f = 0$. I tako dalje. Ide se redom. To je vazno napomenuti. Ovde je kroz 2 primera objasnjeno kako se ovo radi. Ovo crtanje talasnih oblika.

Zadatak: Na ulazima kombinacione mreže sa slike postavljene su vrednosti $(a, b, c) = (0, 1, 1)$. U trenutku $t = 0 \text{ ns}$, menja se vrednost jednog od ulaza. U kom trenutku se menja vrednost izlaza ako se promenila vrednost: *i)* ulaza a , *ii)* ulaza b , *iii)* ulaza c ? Propagaciona kašnjenja logičkih kola su naznačena na slici.

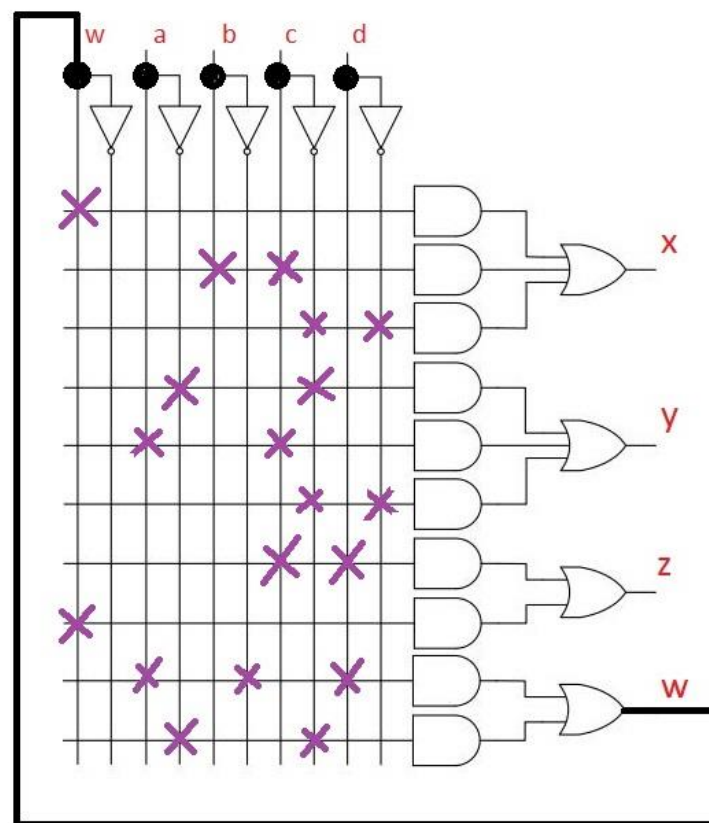
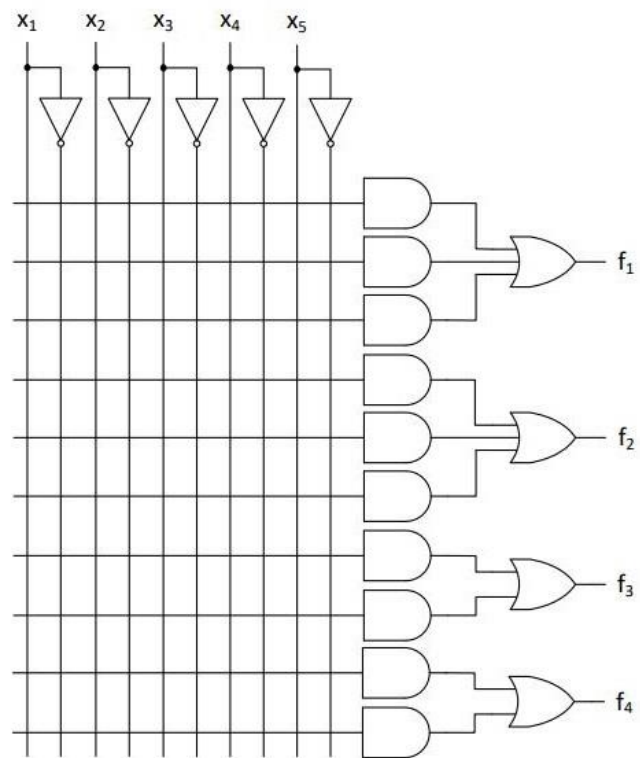


Zadatak: Skup od tri logičke funkcije četiri promenljive realizovati u PAL sa slike (naznačiti spojeve u programabilnim mrežama)

$$x = a\bar{b}d + \bar{a}\bar{c} + bc + \bar{c}\bar{d}$$

$$y = \bar{a}\bar{c} + ac + \bar{c}\bar{d}$$

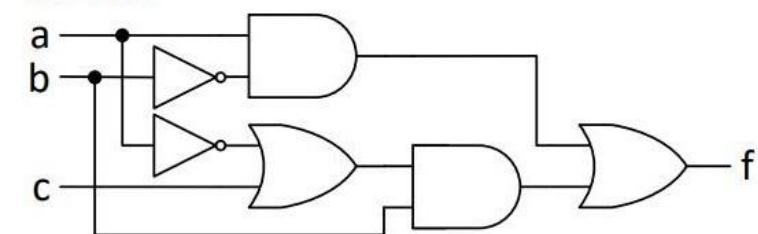
$$z = cd + \bar{a}\bar{c} + a\bar{b}d$$



$$\begin{aligned} x &= \overbrace{a\bar{b}d}^w + \bar{a}\bar{c} + bc + \bar{c}\bar{d} \\ y &= \bar{a}\bar{c} + ac + \bar{c}\bar{d} \\ z &= cd + \underbrace{\bar{a}\bar{c} + a\bar{b}d}_w \end{aligned}$$

Ovde je mnogo vazno napomenuti da smo morali da ovako radimo jer, prost razlog, x ima 4 produktna clana, a u datom PAL-u sa slike to x ima 3 ulaza. Dakle, 4 produktna clana su, i 3 ulaza, i to ne moze. Zato smo morali da uvedemo novu promenljivu w, koja se sastoji od zbira vec postojećih produktnih clanova. Eto to je to. Uvek tako radimo kada postojeći PAL ne moze da ispuni zadatak skup od n logičkih funkcija.

Zadatak: Kombinacionu mrežu koja je prikazana na slici realizovati u obliku kompleksnog CMOS logičkog kola.

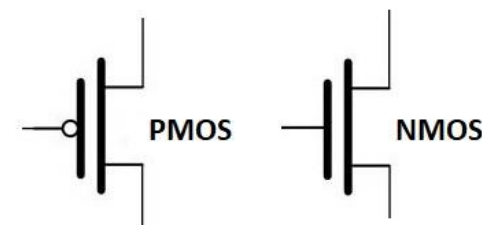
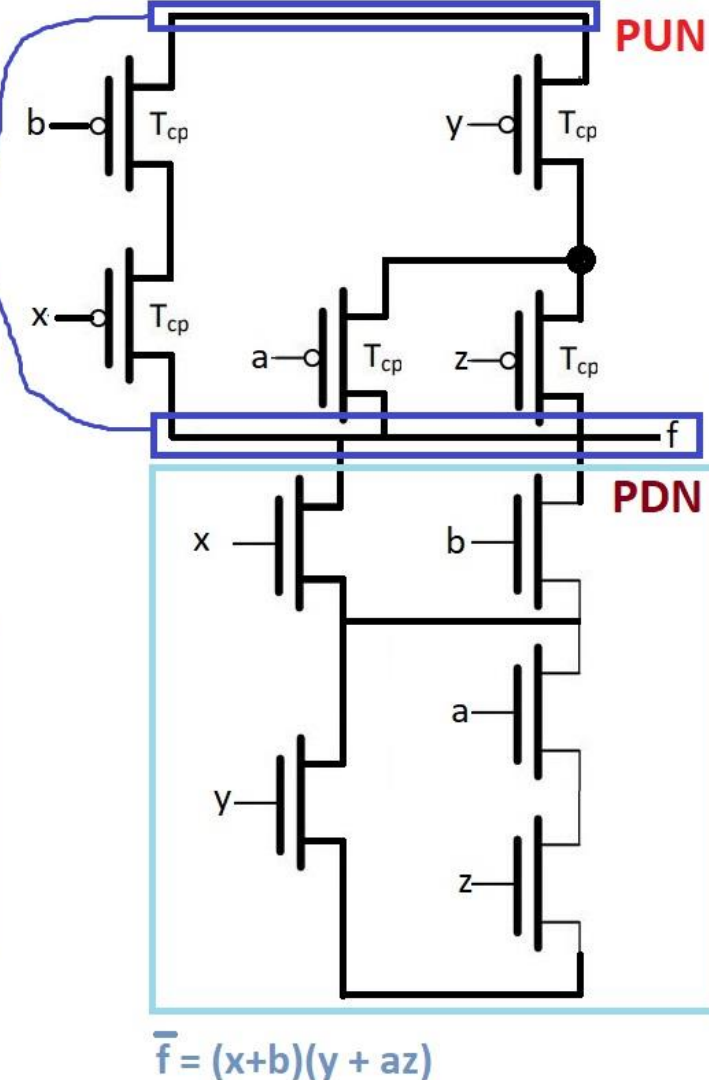
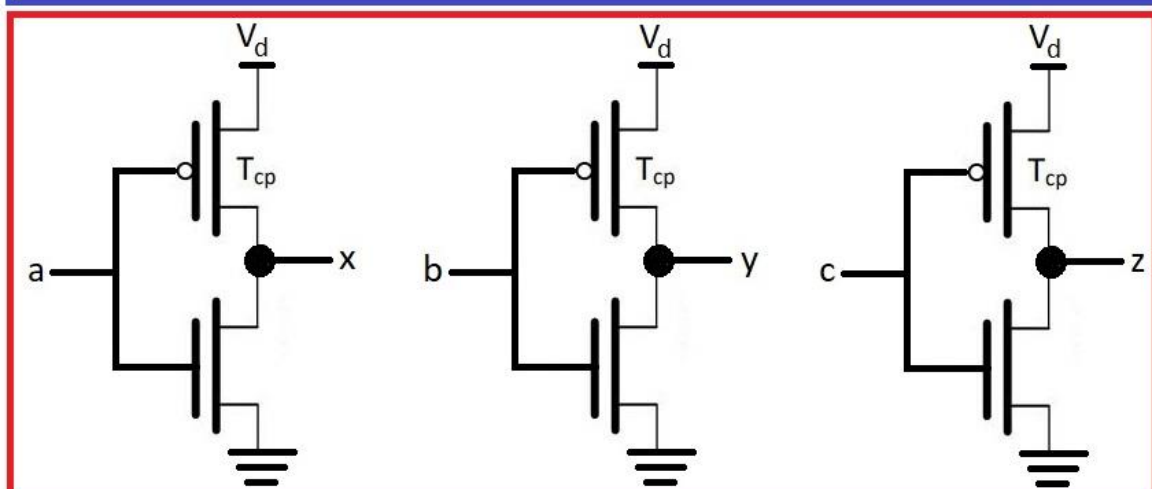


KM realizuje funkciju: $f = a\bar{b} + b(\bar{a} + c) = \bar{x}\bar{b} + \bar{y}(\bar{a} + \bar{z})$

Sta je ovde poenta? Hajde da pojasnimo. Pa nama treba da nam sve promenljive u izrazu budu komplementirane da bi realizovali f. E sad, posto u startnom izrazu za f nisu sve komplementirane promenljive, vec neke jesu, neke nisu, zato uvodimo smenu, tacnije $a = \bar{x}$, $b = \bar{y}$, $c = \bar{z}$ i to realizujemo na nacin na koji je to prikazano u crvenom pravougaoniku. Eto to je poenta cele price. S druge pak strane, ako nijedna promenljiva nije komplementirana, onda radimo komplement citavog izraza i stavljamo inverter na kraju.

Hajde da pojasnimo i ovo. Ovo je paralelna veza i to je zapravo ovaj plus koji se nalazi u izrazu za f nakon uvođenja smene. To je ovaj izraz tamnoplavo napisan ispod ovoga teksta. To je bilo vazno reci. Znacni: $\bar{x}\bar{b} + \bar{y}(\bar{a} + \bar{z})$ - ovo je f

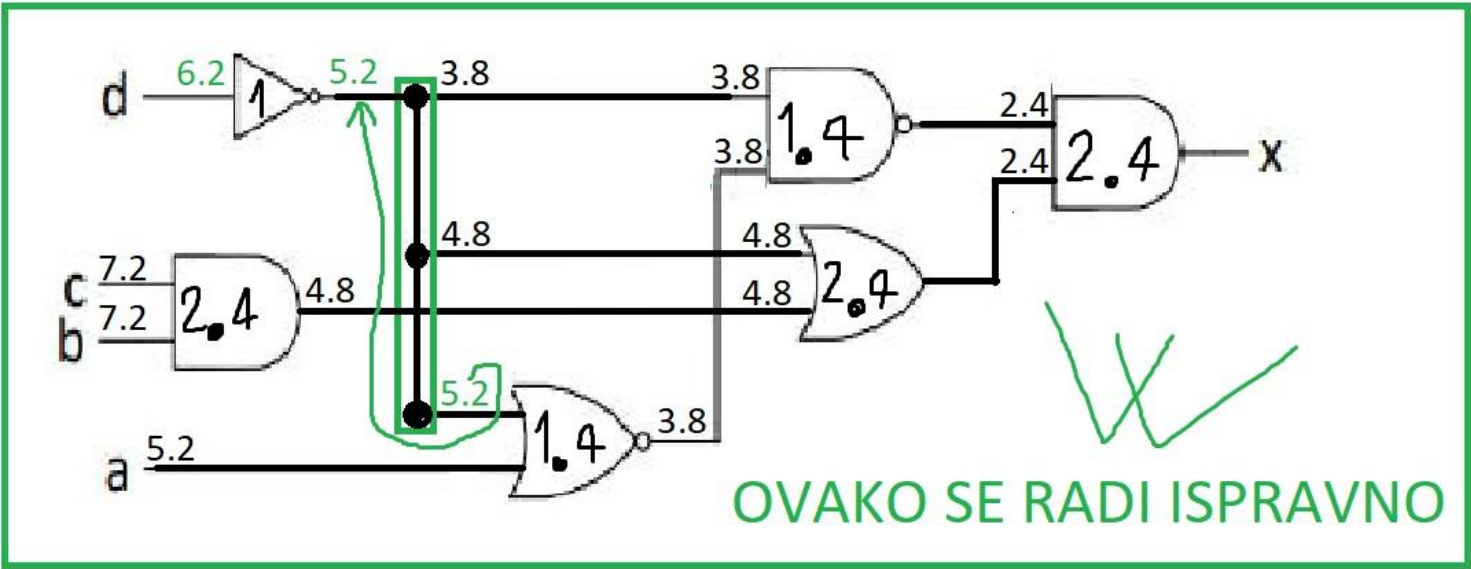
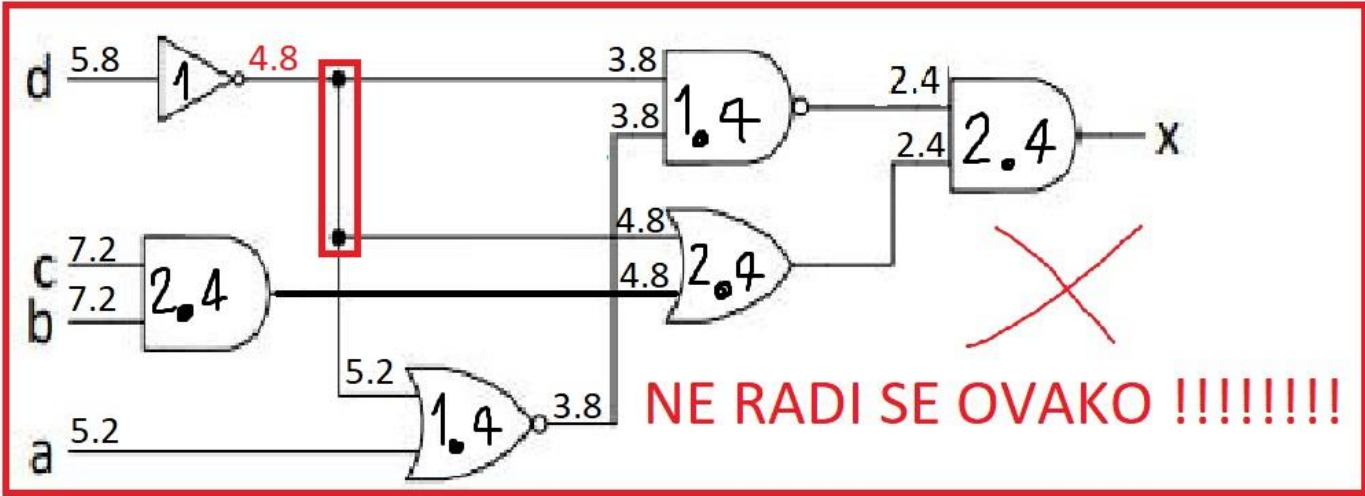
Nakon uvođenja smene



Zadatak: Odrediti propagaciono kašnjenje od svakog ulaza do izlaza kombinacione mreže sa slike ako su propagaciona kašnjenja logičkih kola: NE – 1 ns, NI i NILI – 1.4 ns i I i ILI – 2.4 ns →

Potrebno je prikazati postupak određivanja propagacionog kašnjenja i popuniti tabelu kašnjenja →

ulaz→izlaz	Propagaciono kašnjenje [ns]
a → x	
b → x	
c → x	
d → x	

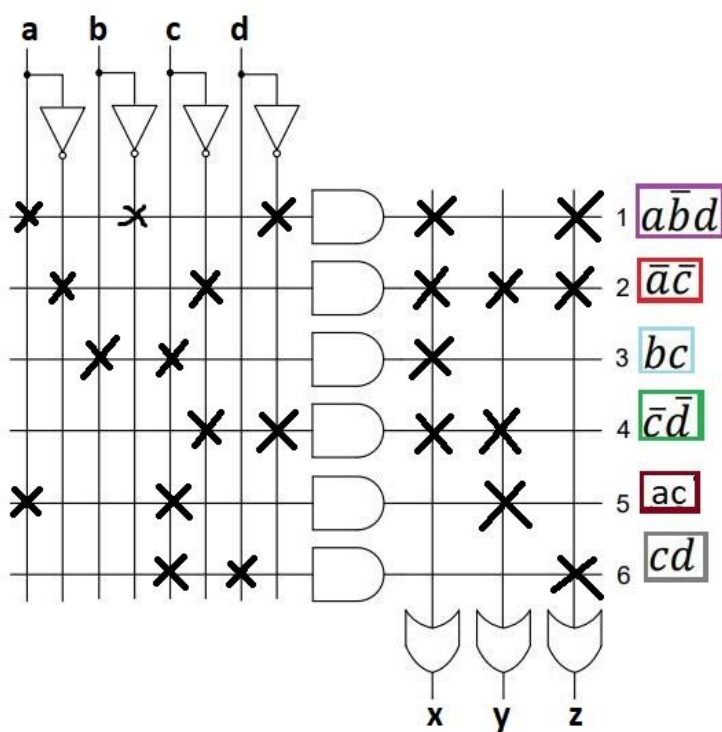
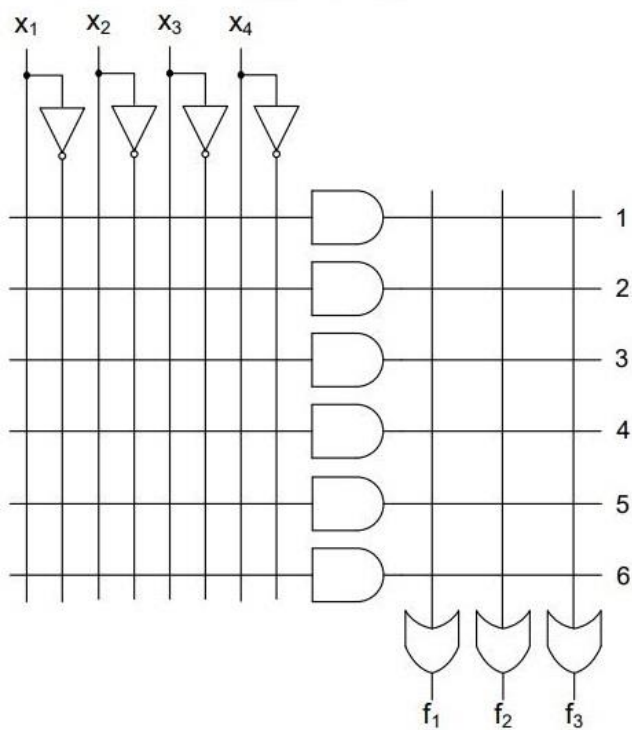


Zadatak: Skup od tri logičke funkcije četiri promenljive realizovati u PLA sa slike (naznačiti spojeve u programabilnim mrežama)

$$x = \boxed{a\bar{b}d} + \boxed{\bar{a}\bar{c}} + \boxed{bc} + \boxed{\bar{c}\bar{d}}$$

$$y = \boxed{\bar{a}\bar{c}} + \boxed{ac} + \boxed{\bar{c}\bar{d}}$$

$$z = \boxed{cd} + \boxed{\bar{a}\bar{c}} + \boxed{a\bar{b}d}$$

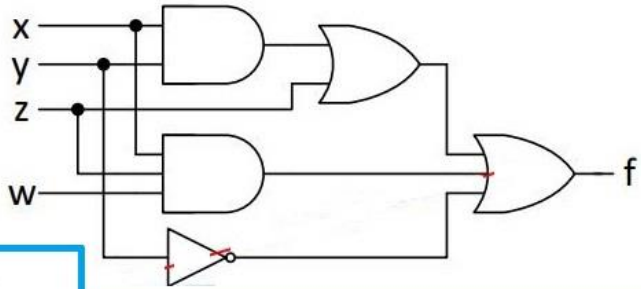


$$x = \boxed{a\bar{b}d} + \boxed{\bar{a}\bar{c}} + \boxed{bc} + \boxed{\bar{c}\bar{d}}$$

$$y = \boxed{\bar{a}\bar{c}} + \boxed{ac} + \boxed{\bar{c}\bar{d}}$$

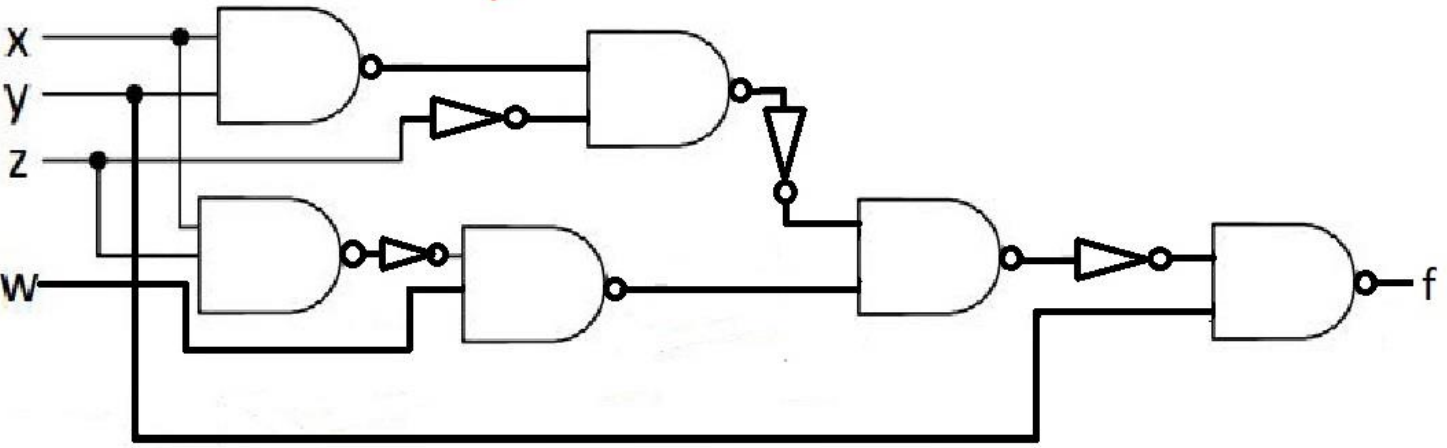
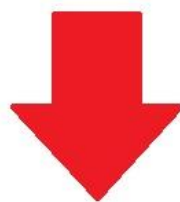
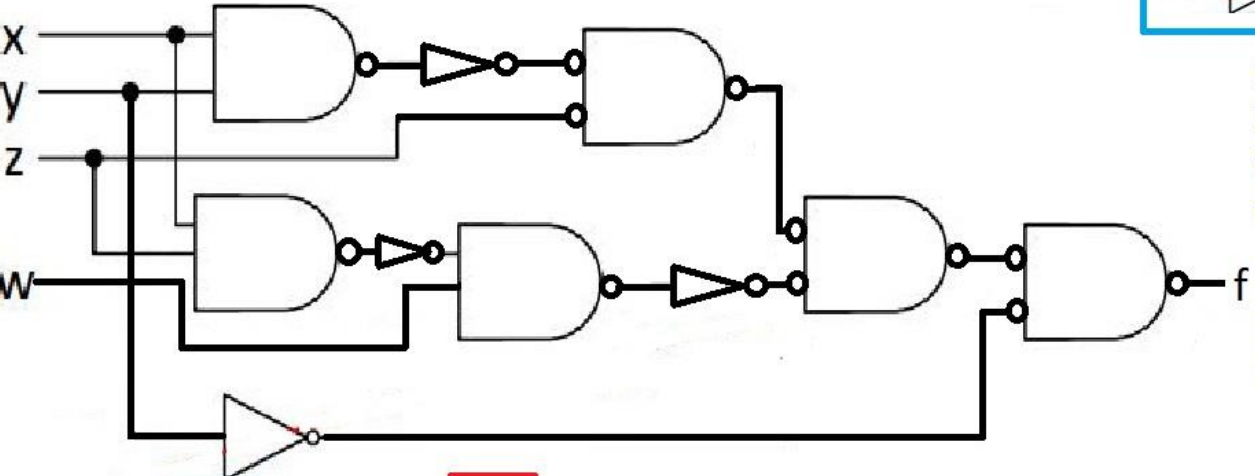
$$z = \boxed{cd} + \boxed{\bar{a}\bar{c}} + \boxed{a\bar{b}d}$$

Zadatak: I-ILI kombinacionu mrežu na slici transformisati u homogenu mrežu dvoulaznih a) NI kola; b) NILI kola

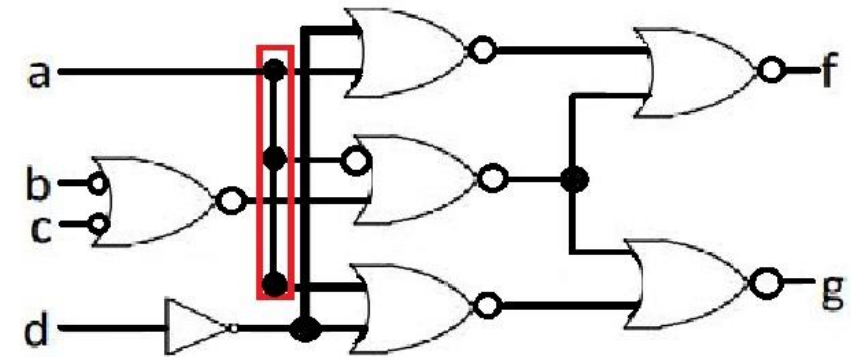
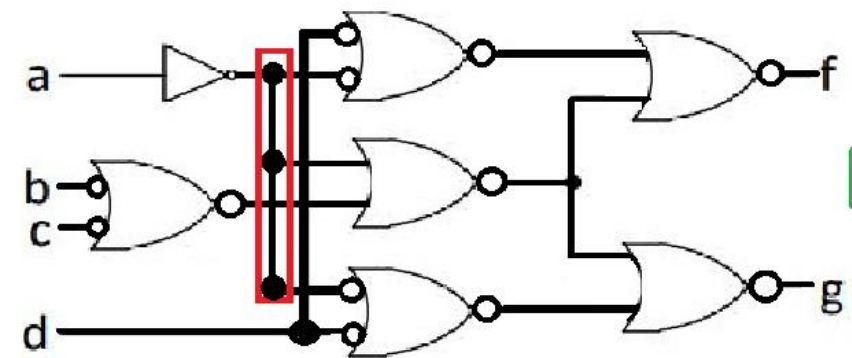
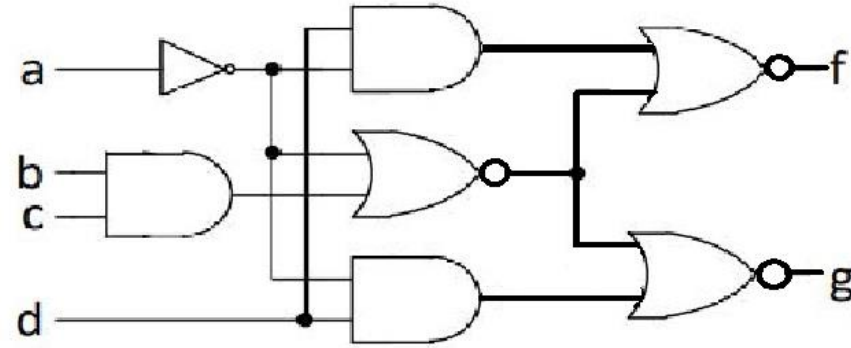
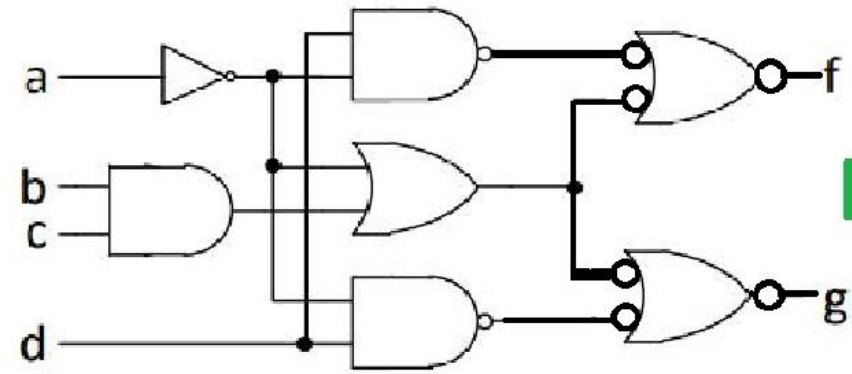
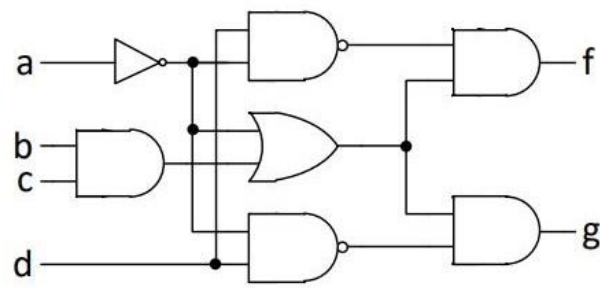


Hajde da pojasnimo poentu ovde. Znaci, posto se trazi da sve bude realizovano preko dvoulaznih NI kola, onda radimo sledece: trebamo da ILI kola stavimo da bude NI sa invertovanim ulazima, a I da bude NI sa invertorima ispred ulaza. To je sve.

Kada dobijemo mrežu, mi onda samo treba da izbacimo suvisne invertore, i onda dobijemo finalnu mrežu, a to je ova na koju pokazuje crvena strelica. To je sve za ovaj zadatak.

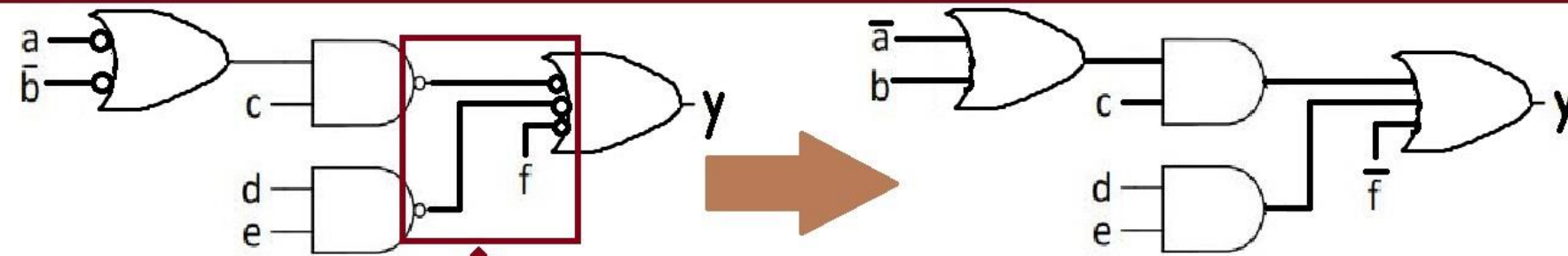
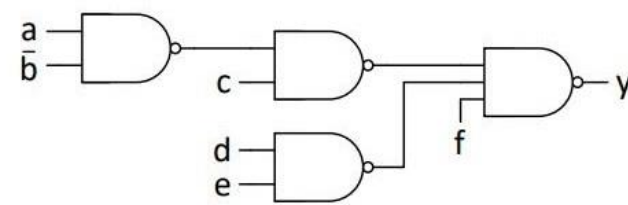


Zadatak: I-ILI kombinacionu mrežu sa slike transformisati u homogenu mrežu logičkih NILI kola.



OVO JE
KONACNO

Zadatak: Homogenu NI kombinacionu mrežu na slici transformisati u I-ILI kombinacionu mrežu. Pretpostavka je da su ulazne promenljive dostupne u pravom i komplementarnom obliku.



KONACNA KOMBINACIONA MREZA

Mi u ovom zadatku krecemo od kraja KM i onda radimo onako nam se dalje uklapa u zavisnosti od toga kakva je mreza. Dakle, polazimo od toga da nam je skroz desno NI kolo, a njega pretvaramo u ILI kolo sa invertovanim ulazima.

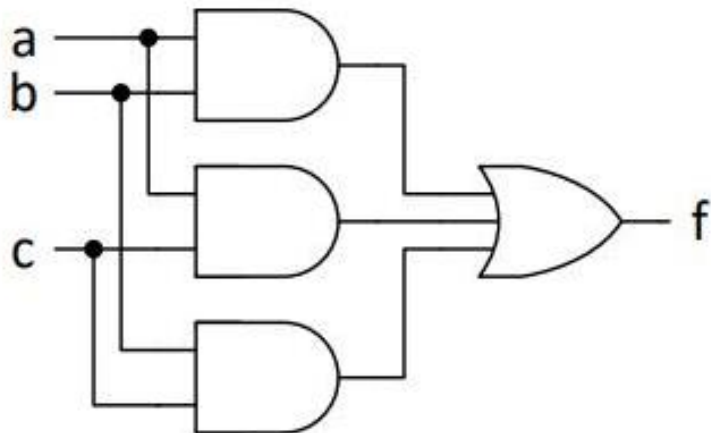
Ustvari, to pretvaranje uvek radimo i onda dobijemo to sto nam treba. Skratimo suvisne invertore i dobijemo konacnu KM. To je sve za ovaj zadatak.

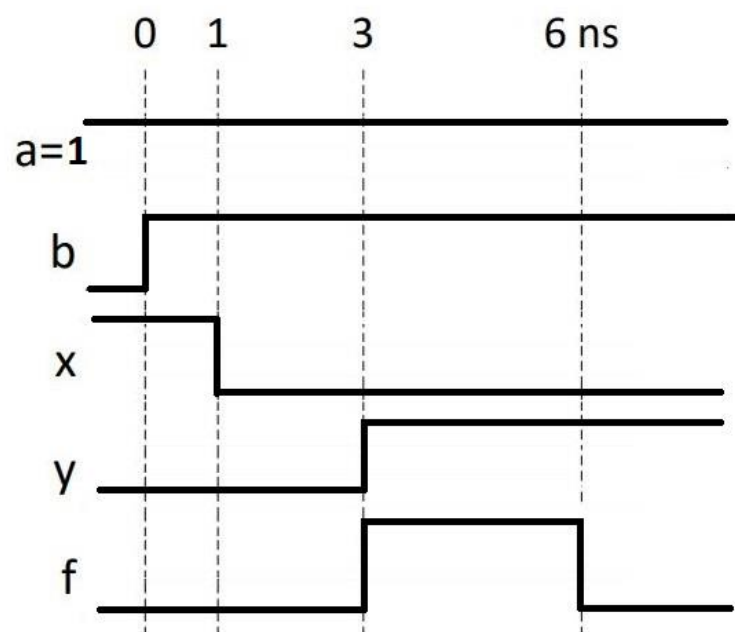
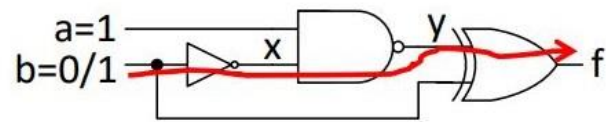
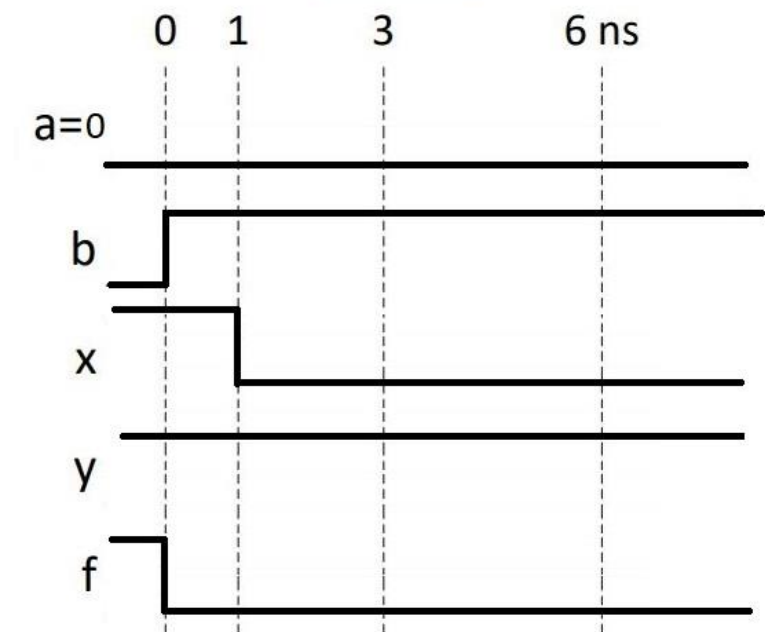
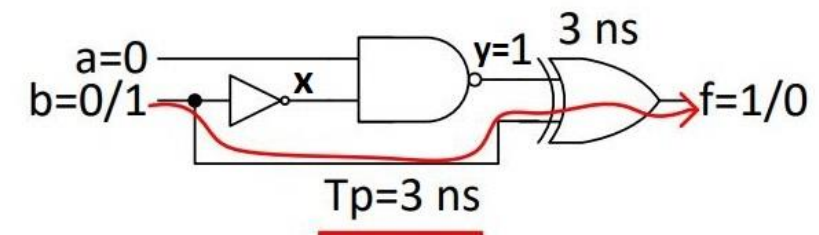
Bankarski sef ima tri brave sa različitim ključem za svaku bravu. Svaki ključ je u vlasništvu različite osobe. Da bi se sef otvorio, bar dve osobe moraju da ubace svoje ključeve u odgovarajuće brave. Signali a , b i c su na nivou 1 ako je ključ ubačen u bravu 1, 2 ili 3. Projektovati kombinatornu mrežu koja će generisati signal f za otvaranje sefa.

a	b	c	z
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

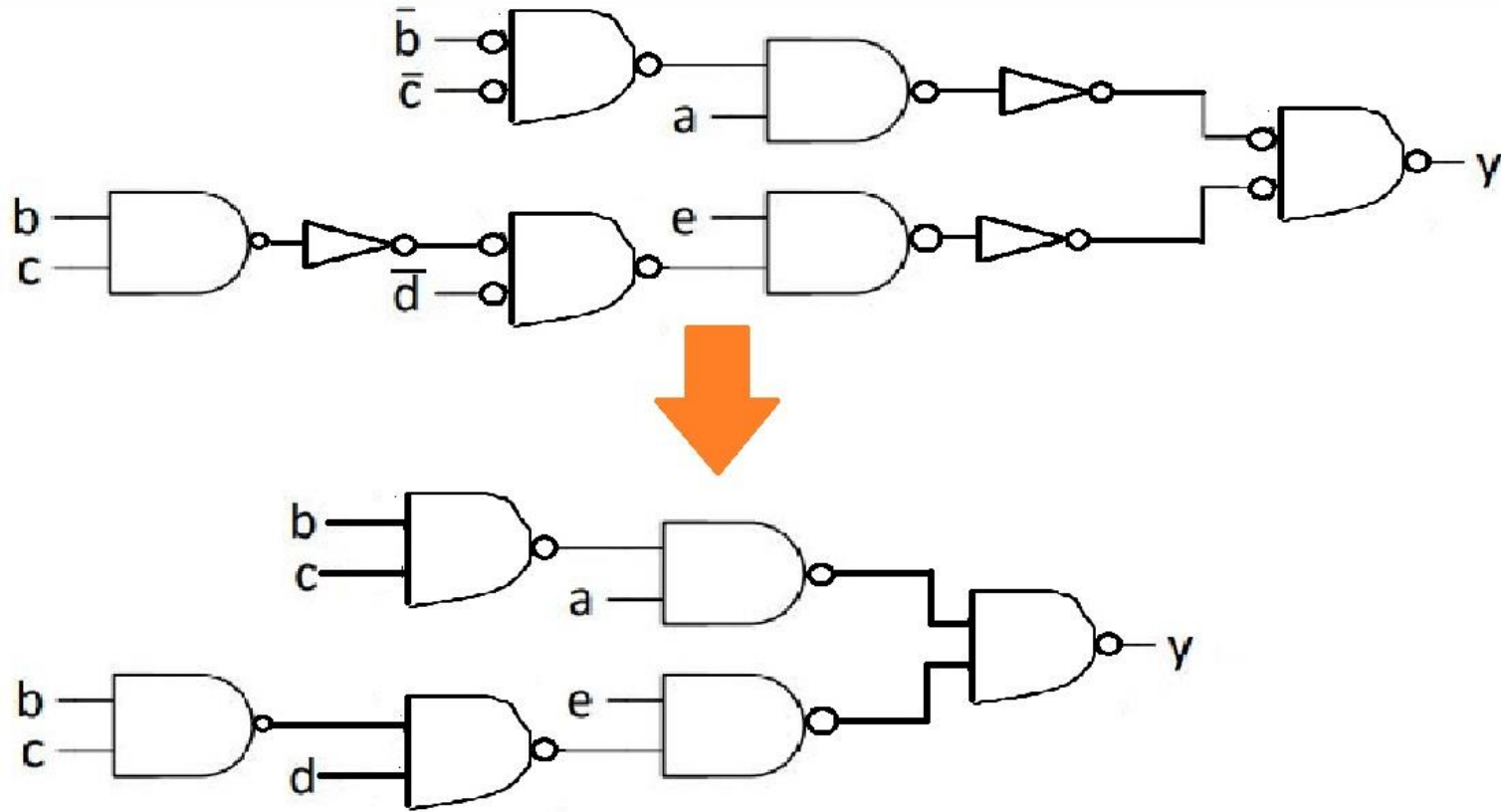
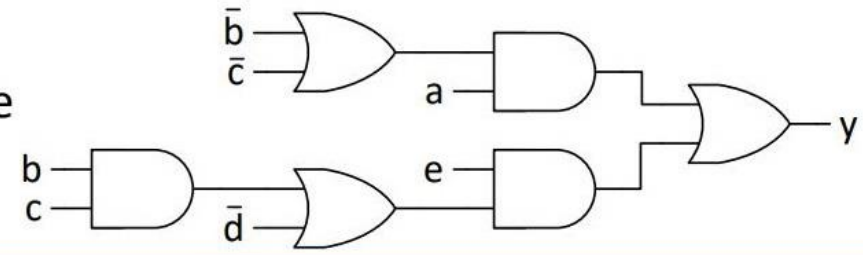
a \ bc	00	01	11	10
0	0	0	1	0
1	0	1	1	1

$z = bc + ac + ab \rightarrow$ ovo je minimizacija Karnoom



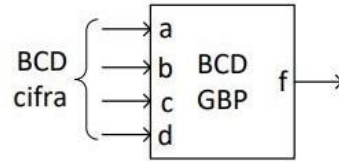


Zadatak: I-II kombinacionu mrežu na slici transformisati u homogenu mrežu dvoulaznih NI kola. (Pretpostavka je da su ulazne promenljive dostupne u pravom i komplementarnom obliku.)



Logičke funkcije

(Primer nepotpuno definisane funkcije) Kombinaciono kolo sa slike je generator neparne parnosti BCD cifre. Na ulaz kola se dovodi BCD cifra, a na izlazu se generiše bit koji svojom vrednošću dopunjuje broj 1-ca na ulazu do neparnog broja. Na primer, bit neparne parnosti BCD cifre 0000_2 je 1, 0001_2 je 0, 0010_2 je 0 itd. Popuniti tabelu istinitosti i izvesti minimalni SOP izraz za funkciju f primenom Karnoove mape.



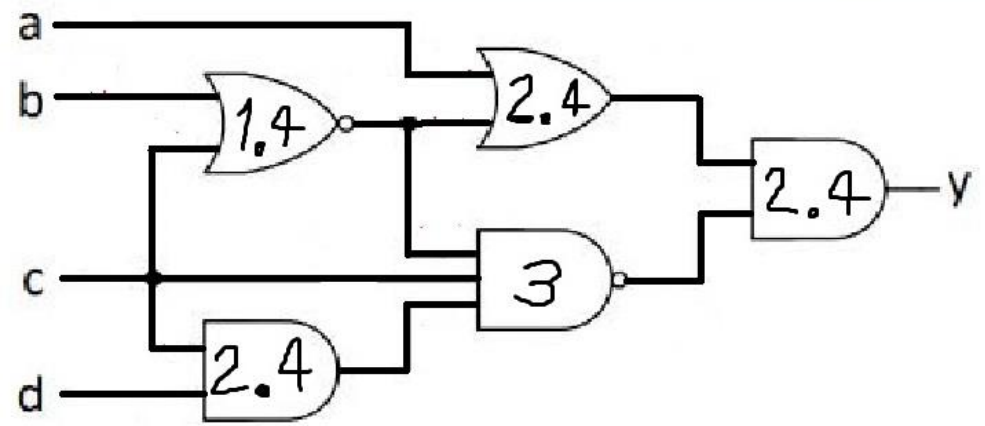
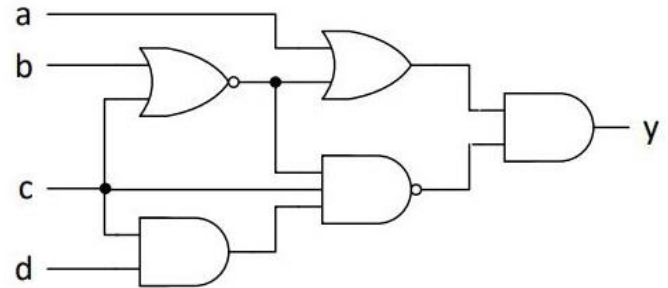
↓

a	b	c	d	f
0	0	0	0	1
0	0	0	1	0
0	0	1	0	0
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	0
1	0	0	0	0
1	0	0	1	1
1	0	1	0	X
1	0	1	1	X
1	1	0	0	X
1	1	0	1	X
1	1	1	0	X
1	1	1	1	X

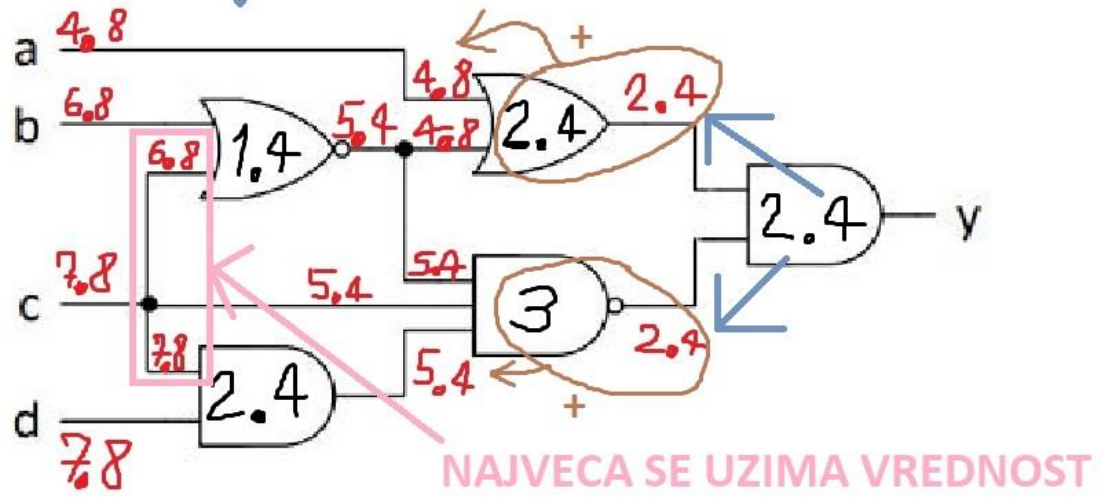
$\begin{matrix} c & d \\ a & b \end{matrix}$	00	01	11	10
00	1		1	
01		1		1
11	X	X	X	X
10		1	X	X

$$f = \bar{a}\bar{b}\bar{c}\bar{d} + \bar{b}cd + b\bar{c}d + bcd\bar{d} + ad$$

Zadatak: Odrediti propagaciono kašnjenje kombinacione mreže sa slike1 ako su propagaciona kašnjenja logičkih kola: dvoulazno NILI – 1.4 ns, dvoulazno I i ILI – 2.4 ns, i troulazno NI – 3 ns.



DOBIJA SE



Zadatak: Primenom zakona o konsenzusu pojednostaviti logički izraz: $a\bar{c}d + ac\bar{d} + b\bar{c}d + bc\bar{d} + a\bar{b}$

$$a\bar{c}d + \underline{ac\bar{d}} + b\bar{c}d + \underline{bc\bar{d}} + \underline{a\bar{b}} \quad \Rightarrow \quad a\bar{c}d + \cancel{ac\bar{d}} + b\bar{c}d + bc\bar{d} + a\bar{b}$$

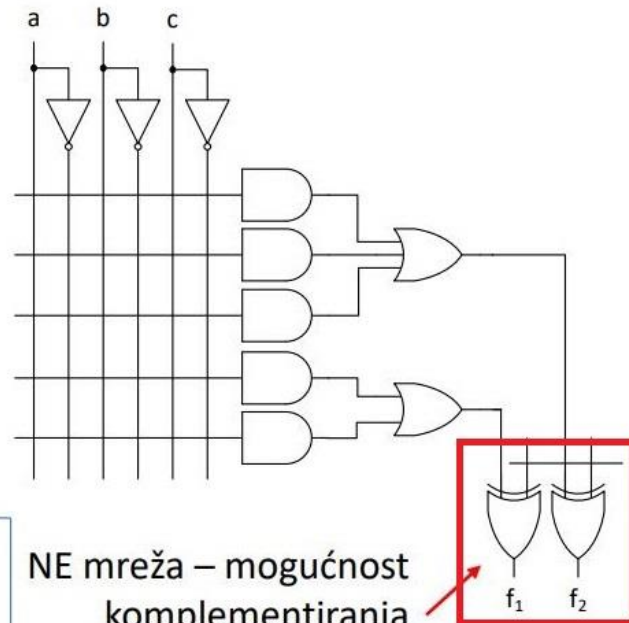
$$a\bar{c}d + \underline{b\bar{c}d} + bc\bar{d} + \underline{a\bar{b}} \quad \Rightarrow \quad \cancel{ac\bar{d}} + b\bar{c}d + bc\bar{d} + a\bar{b}$$

$$b\bar{c}d + bc\bar{d} + a\bar{b}$$

Zadatak: Pojednostaviti logički izraz: $(\overline{a}(\overline{b + c}))(\overline{a + b + c})(\overline{abc})$

$$\begin{aligned} (a + \overline{b}c)(\overline{a + b + c})(\overline{a + b + c}) &= (a + \overline{b})(a + c)(\overline{b + c} + \overline{a \cdot a}) = (a + \overline{b})(a + c)(\overline{b + c}) \\ &= \cancel{(a + \overline{b})}(a + c)(\overline{b + c}) = (\overline{b + c})(a + c) \end{aligned}$$

Zadatak: Logičke funkcije tri promenljive: $y = a\bar{b}\bar{c} + \bar{a}c + \bar{a}b + bc$ i $z = c + a\bar{b} + \bar{a}b$ realizovati pomoću PAL sa slike



Za svaku od dve funkcije y i z postoje dve mogućnosti:

1) Realizuje se u I-ILI mreži i propušta kroz NE mrežu bez promene

2) U I-ILI mreži se realizuje komplement funkcije koji se dodatno komplementira u NE mreži.

Za slučaj 2) potrebno je odrediti:

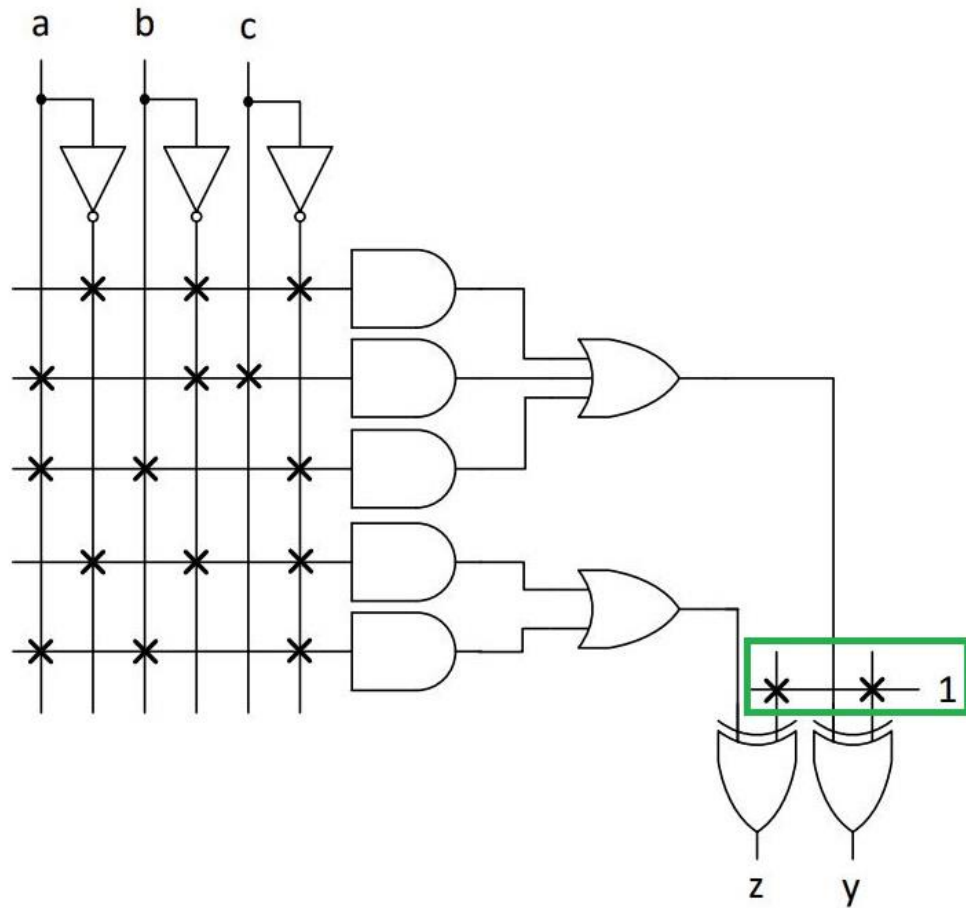
NE mreža – mogućnost komplementiranja izlaza ILI mreže

Za realizaciju biramo $\bar{y}$ i $\bar{z}$ jer se uklapaju u dimenzije PAL kola

$$\begin{aligned}\bar{y} &= \overline{a\bar{b}\bar{c} + \bar{a}c + \bar{a}b + bc} \\ &= (\bar{a} + b + c)(a + \bar{c})(a + \bar{b})(\bar{b} + \bar{c}) \\ &= (b(a + \bar{c}) + (\bar{a} + c)(a + \bar{c}))(\bar{b} + a\bar{c}) \\ &= (ab + b\bar{c} + \bar{a}\bar{c} + ac)(\bar{b} + a\bar{c}) \\ &= \bar{a}\bar{b}\bar{c} + \bar{a}\bar{b}c + \bar{a}b\bar{c}\end{aligned}$$

$$\begin{aligned}\bar{z} &= \overline{c + a\bar{b} + \bar{a}b} \\ &= \bar{c}(\bar{a} + b)(a + \bar{b}) \\ &= \bar{c}(\bar{a}a + \bar{a}\bar{b} + ab + b\bar{b}) \\ &= \bar{c}(\bar{a}\bar{b} + ab) \\ &= \bar{a}\bar{b}\bar{c} + \bar{a}b\bar{c}\end{aligned}$$

Sa izvedenim izrazima za $\bar{y}$ i $\bar{z}$, kandidati za realizaciju su sledeće četiri para funkcija (y, z) , $(y, \bar{z})$, $(\bar{y}, z)$ i $(\bar{y}, \bar{z})$, od kojih se samo poslednji par uklapa u dimenzije PAL sa Sl. 2-24. Na Sl. 2-25 je prikazana realizacija $(\bar{y}, \bar{z})$.



sta ovo predstavlja sa ovom jedinicom?

Zadatak: Logički izraz $(\bar{b} + c)(\bar{a} + \bar{c} + \bar{d})(a + b + \bar{c})$ transformisati u što jednostavniji SOP oblik.

$$\begin{aligned}(\bar{b} + c)(\bar{a} + \bar{c} + \bar{d})(a + b + \bar{c}) &= (\bar{b} + c)(\bar{c} + \bar{a} + \bar{d})(\bar{c} + a + b) = \\(\bar{b} + c)(\bar{c} + (\bar{a} + \bar{d})(a + b)) &= (\bar{b} + c)(\bar{c} + \bar{a}b + a\bar{d} + \bar{d}b) = \\ \bar{b}\bar{c} + a\bar{b}\bar{d} + \bar{a}bc + a\bar{c}\bar{d} + bc\bar{d} &= \bar{b}\bar{c} + a\bar{b}\bar{d} + \bar{a}bc + a\bar{c}\bar{d} + bc\bar{d} = \\ a\bar{b}\bar{d} + \bar{b}\bar{c} + \bar{a}bc + a\bar{c}\bar{d} &= \bar{b}\bar{c} + \bar{a}bc + a\bar{c}\bar{d} = \bar{b}\bar{c} + c(\bar{a}b + \bar{d}) =\end{aligned}$$

Zadatak: Logički izraz $\bar{a}\bar{b} + bcd + a\bar{c}\bar{d} + a\bar{b}\bar{c}$ svesti na oblik proizvod suma $(x + x)(x + x + x)(x + x + x)$

$$\begin{aligned}\bar{a}\bar{b} + bcd + a\bar{c}\bar{d} + a\bar{b}\bar{c} &= \bar{b}(\bar{a}\bar{c} + \bar{a}) + d(\bar{a}\bar{c} + bc) = (\bar{b} + \bar{d})(\bar{b} + \bar{a}\bar{c} + bc)(\bar{a}\bar{c} + \bar{a} + \bar{d})(\bar{a}\bar{c} + \bar{a}\bar{c} + \bar{a} + bc) \\ &= (\bar{b} + \bar{d})(\bar{b} + c + \bar{a}\bar{c})(\bar{d} + \bar{c} + \bar{a})(\bar{a}\bar{c} + \bar{a} + bc) = (\bar{b} + \bar{d})(\bar{b} + c + \bar{a})(\bar{d} + \bar{c} + \bar{a})(\bar{c} + \bar{a} + b) = \\ &= (\bar{b} + \bar{d})(\bar{b} + c + \bar{a})\cancel{(\bar{d} + \bar{c} + \bar{a})}(\bar{c} + \bar{a} + b) = (b + \bar{c} + \bar{a})(\bar{b} + \bar{d})(\bar{b} + c + a)\end{aligned}$$

Zadatak: Primenom algebarskih transformacija pojednostaviti logički izraz: $(a + b + \bar{c})(a + b + d)(a + b + e)(a + \bar{d} + e)(a + \bar{c})$. Rezultujući logički izraz može biti proizvoljnog oblika (SOP, POS ili slobodna forma), ali sa ne više od 6 literala.

$$\begin{aligned}
 & (a+b+\bar{c})(a+b+d)(a+b+e)(a+\bar{d}+e)(a+\bar{c}) = \\
 & (a+b+\bar{c})(a+b+d)(a+b+e)(a+\bar{d}+e)(a+\bar{c}) = \\
 & \underline{(a+b+\bar{c})(a+b+d)(a+\bar{d}+e)(a+\bar{c})} = \\
 & \underline{(a+b+\bar{c})(a+b+d)(a+\bar{d}+e)(a+\bar{c})} = \\
 & (a+b+d)(a+\bar{d}+e)(a+\bar{c}) = \\
 & (a+b+d)(a+\bar{d}+e)(a+\bar{c}) = \\
 & a + (b+d)(\bar{d}+e)\bar{c} = \\
 & a + (\bar{c}(b\bar{d} + be + de)) = \\
 & a + (\bar{c}(b\bar{d} + be + de)) = \\
 & a + \bar{c}b\bar{d} + \bar{c}de = \\
 & a + \bar{c}(b\bar{d} + de)
 \end{aligned}$$

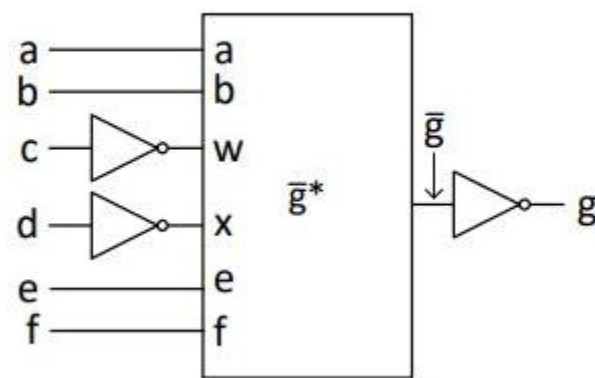
↓ преобразован результат и имеет 7 литералов, а может быть max 6

Zadatak: Koliko MOS tranzistora je potrebno za realizaciju log. funkcije $g = (a + b\bar{c})(\bar{d} + e + f)$ u vidu kompleksnog CMOS logičkog kola?

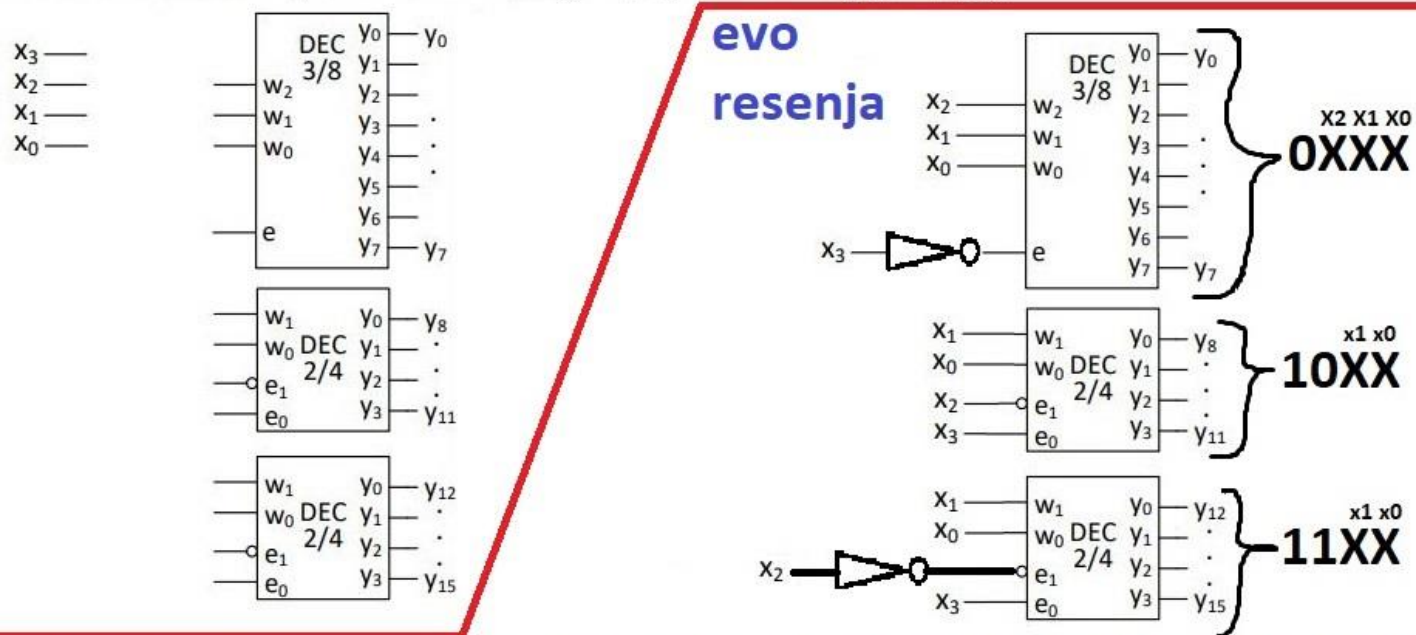
Rešenje : Realizovati CMOS kolo za funkciju $\bar{g}$ i invertovati izlaz.

$$\bar{g} = \overline{(a + b\bar{c})(\bar{d} + e + f)} = \bar{a}(\bar{b} + c) + d\bar{e}\bar{f}$$

Potrebno je: 2x2=4 tranzistora za ulazne invertore i 2x6=12 za PUN i PDN mreže, odnosno ukupno **18 tranzistora**.



Zadatak 3-5: Pomoću binarnih dekodera sa Sl. 3-9 i minimalnog broja dodatnih logičkih kola formirati binarni dekodер 4/16 sa ulazima $x_3, \dots, x_0$ i izlazima $y_0, \dots, y_{15}$.



Hajde da objasnimo kako radi dekodер. Znaci, vazno je reci da su tipovi dekodera 2/4, zatim 3/8 itd. Dakle, $n/2^n$. Okej, to je prvi deo zadatka. To se mora zna za dekodер. Drugi deo je vazan isto, a to je da je receno da su ulazi od x_3 do x_0 . Ovaj deo sto nam se povezuje sa w_0, w_1, w_2 itd. to nam ide x_0, x_1, x_2 itd. Dakle prikljucuje se na to. I da, ovi veliki X-ovi sto su nam desno od dekodera, i tu mora kazemo da nam je skroz desno x_0 , pa x_1 i sad zavisi koliko njih imamo, tolko cemo tih malih x-ova da stavimo iznad velikih X-ova. Sve bas kao sto je naznaceno na slici. Dakle od sustinske je vaznosti reci da nam je skroz desno x_0 , pa x_1 itd. To je sa desne strane. A sa leve, odozdo nagore idu x_0 , pa x_1 , pa x_2 eventualno itd. Sve opet zavisi od slucaja do slucaja. A ovo sto nam je za dozvolu rada, za to je obrnuto, to ide odozgo nadole. Znaci, aj da ponovimo, ovo sto nam ide desno to su ovi veliki X-ovi i ide nam zdesna nalevo x_0 ...pa nadalje. To nam se sa leve strane vezuje sa w_0 ...pa nadalje odozdo nagore. Ovo nadalje, sto vise nije X veliko, nego jedinice i nule, e to nam sa leve strane ide kontra, odozgo nadole. A taj raspored jedinica i/ili nula, to nam odredjuju ovi indeksi pored y . Ali ovi desno od ove linije, a ne unutar dekodera. To je citavo objasnjenje za ovaj zadatak.

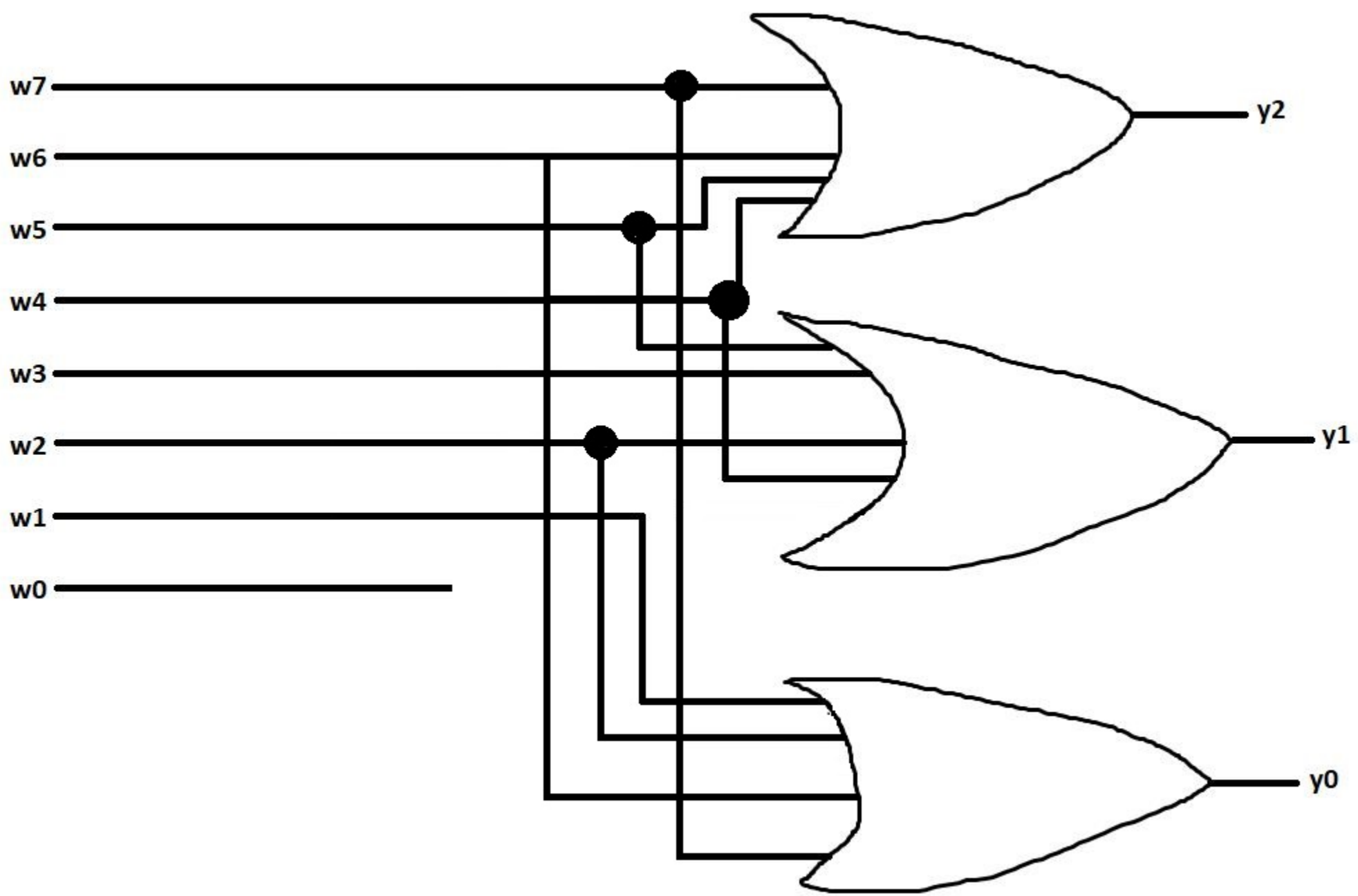
Zadatak: Projektovati koder 8/3 za Grejov kôd, prema tabeli istinitosti:

x7	x6	x5	x4	x3	x2	x1	x0	y2	y1	y0
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	1	0	0	0	1	1
0	0	0	0	1	0	0	0	0	1	0
0	0	0	1	0	0	0	0	1	1	0
0	0	1	0	0	0	0	0	1	1	1
0	1	0	0	0	0	0	0	1	0	1
1	0	0	0	0	0	0	0	1	0	0

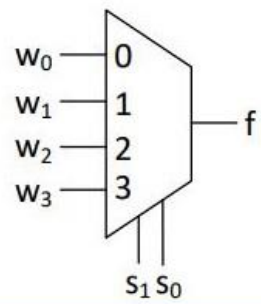
Poenta kod Grejovog koda je ta da imamo u startu 000, pa zatim 001, i onda nadalje menjamo bitove. E sad, sta je bitno reci? Pa to da mi menjamo samo 1 bit u odnosu na prethodni izgled bitova i to gledamo da promenimo krajnji desni, ali ako dobijamo neku kombinaciju koju vec imamo onda menjamo drugi zdesna, tj. onaj koji je odmah pored krajnjeg desnog, itd.

Sada cemo objasniti drugi deo zadatka, a to je realizacija logickim kolima. Dakle ovako: mi za y2 imamo da mu je vrednost 1 samo ako je x4, x5, x6, i x7 jednako 1. I onda znaci mi na ulaz stavljamo te bitove a na izlaz Ili kola ide y2. Tako radimo i za ovo ostalo.

KODER JE NA SLEDECEM SLAJDU



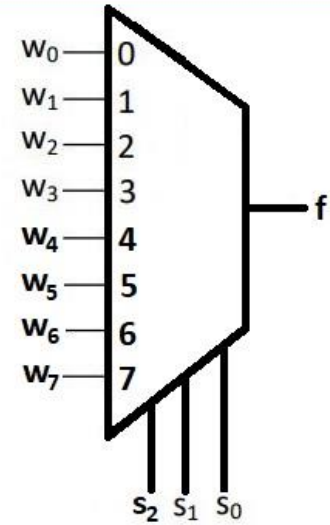
MUX 4/1



s_1	s_0	f
0	0	w_0
0	1	w_1
1	0	w_2
1	1	w_3

$$f = \overbrace{\bar{s}_1 \bar{s}_0}^{m_0} w_0 + \overbrace{\bar{s}_1 s_0}^{m_1} w_1 + \overbrace{s_1 \bar{s}_0}^{m_2} w_2 + \overbrace{s_1 s_0}^{m_3} w_3$$

MUX 8/1

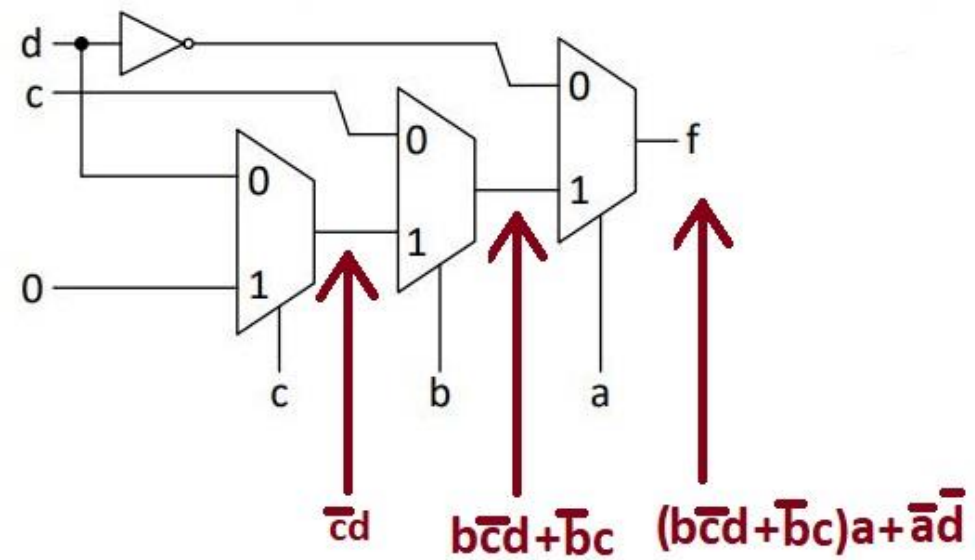


s_2	s_1	s_0	f
0	0	0	w_0
0	0	1	w_1
0	1	0	w_2
0	1	1	w_3
1	0	0	w_4
1	0	1	w_5
1	1	0	w_6
1	1	1	w_7

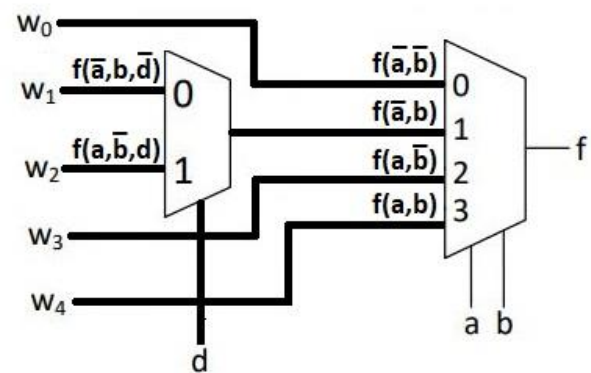
Hajde da pojasnimo rad multipleksera. Dakle ovako, MUX $2^n / 1$ ima 2^n ulaza i n selekcionih ulaza, koji su raspoređeni tako da je s_0 skroz desno, pa s_1 odma pored njega, drugo mesto zdesna, itd. Tako su raspoređeni selekcionih ulazi, a ovi obični ulazi su raspoređeni odozgo nadole. Imamo skroz gore w_0 , pa w_1 , pa w_2 itd. A imamo i brojeve od 0 do $2^n - 1$, koji su takodje odozgo nadole.

$$f = \bar{s}_2 \bar{s}_1 \bar{s}_0 w_0 + \bar{s}_2 \bar{s}_1 s_0 w_1 + \bar{s}_2 s_1 \bar{s}_0 w_2 + \bar{s}_2 s_1 s_0 w_3 + s_2 \bar{s}_1 \bar{s}_0 w_4 + s_2 \bar{s}_1 s_0 w_5 + s_2 s_1 \bar{s}_0 w_6 + s_2 s_1 s_0 w_7$$

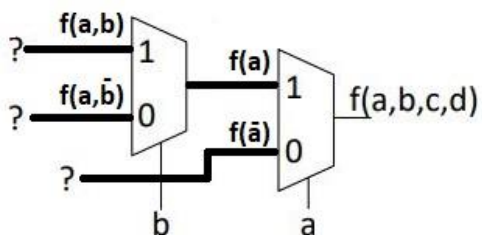
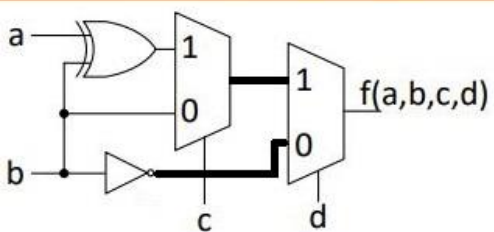
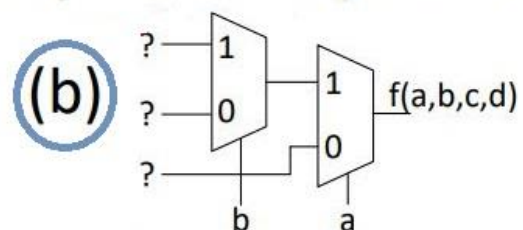
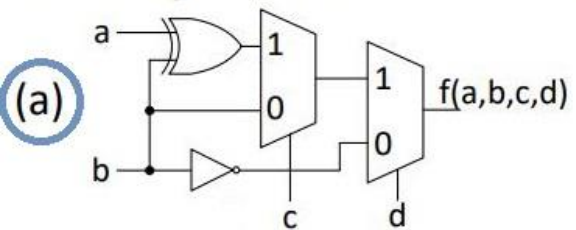
Odrediti logičku funkciju koju realizuje multiplekstersko kolo sa slike. Rešenje dati u obliku SOP izraza.



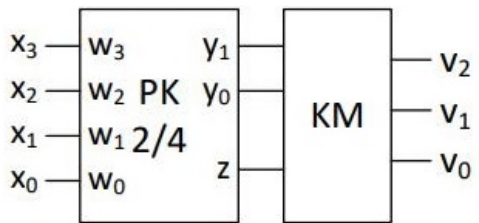
Logičku funkciju $f = \bar{a}\bar{b}c + b\bar{c}d + ac + bcd$ realizovati u multipleksterskoj mreži sa slike upotrebom što manjeg broja dodatnih logičkih kola.



Odrediti logičku funkciju $f(a, b, c, d)$ koju realizuje multipleksterska mreža sa (a), a zatim istu funkciju realizovati u multipleksterskoj mreži sa (b).



Zadatak: Prema dijagramu na slici projektovati kolo za prebrojavanje „vodećih“ (početnih) nula 4-bitnog niza $X = x_3x_2x_1x_0$. (Blok PK predstavlja prioritetni koder, a blok KM kombinacionu mrežu.) Rezultat prebrojavanja je prisutan na izlazu $V = v_2v_1v_0$ u obliku 3-bitnog binarnog broja u opsegu $0 - 4$. Na primer, u nizu $X = 1010$ ima $V = 0$ vodećih nula; broj vodećih nula u nizu $X = 0011$ je $V = 2$, dok za niz $X = 0000$ važi $V = 4$.



x3	x2	x1	x0	y1	y0	z	v2	v1	v0
0	0	0	0	0	0	0	1	0	0
0	0	0	1	0	0	1	0	1	1
0	0	1	X	0	1	1	0	1	0
0	1	X	X	1	0	1	0	0	1
1	X	X	X	1	1	1	0	0	0

y1 \ y0 z	00	01	11	10
0	0	1	1	0
1	0	0	0	0

$$v_1 = \bar{y}_1 z$$

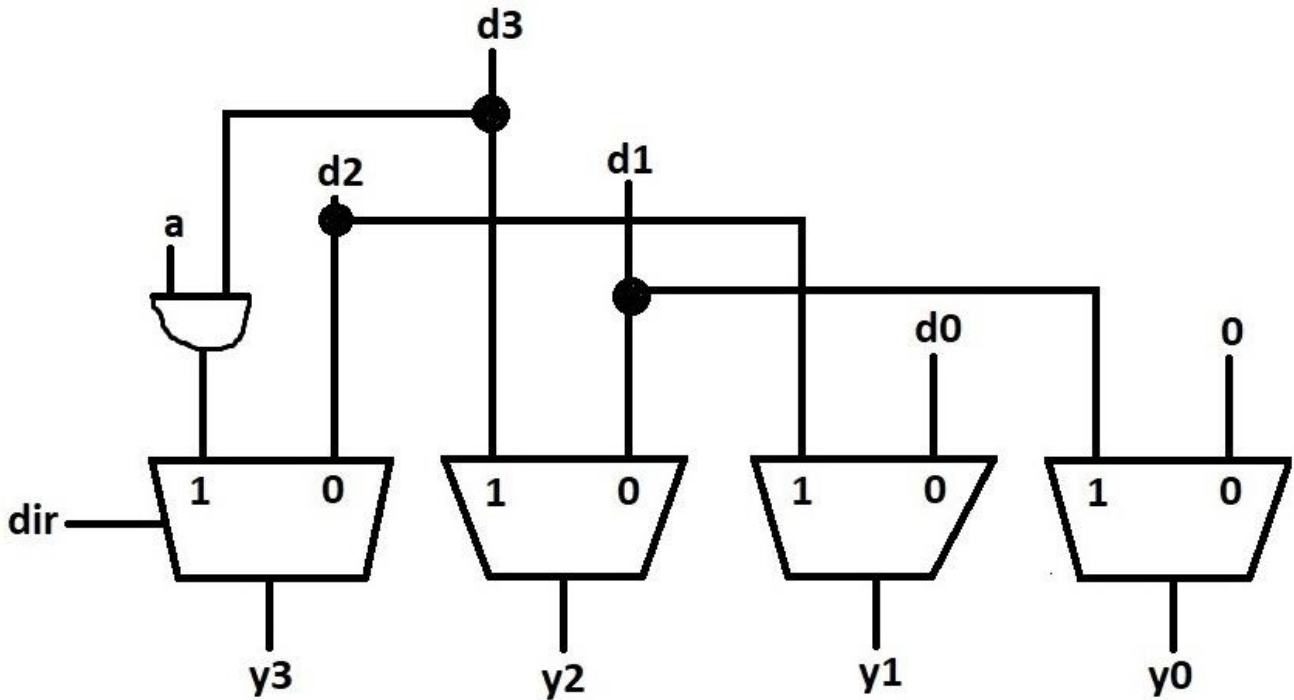
y1 \ y0 z	00	01	11	10
0	0	1	0	0
1	0	1	0	0

$$v_0 = \bar{y}_0 z$$

Resenje Karnoovom mapom

Da razjasnimo. Znaci, koder (obicni koder) ima 1-ce dijagonalno, i samo njih, i nema signal dozvole i iksove, a prioritetni koder ima signal dozvole i ima x-ove, i to ispod tih keceva. Ceo taj red su nule kada je $e = 0$, i to valja napomenuti da vazi za prioritetni koder. Kod obicnog kodera nema toga. Eto to je sve, cela napomena.

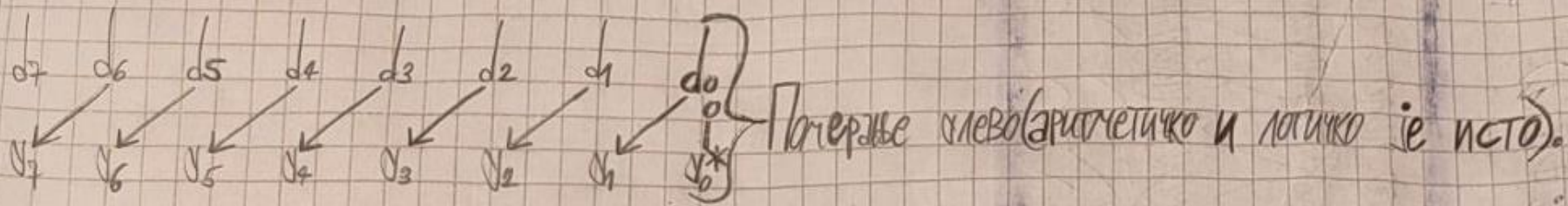
a	dir	y ₃	y ₂	y ₁	y ₀	F-ja
0	0	d ₂	d ₁	d ₀	0	logičko pomeranju ulevo
0	1	0	d ₃	d ₂	d ₁	logičko pomeranje udesno
1	0	d ₂	d ₁	d ₀	0	aritmetičko pomeranje ulevo
1	1	d ₃	d ₃	d ₂	d ₁	aritmetičko pomeranje udesno



Dakle, hajde da objasnimo o čemu se ovde radi. Kao što smo rekli, kritične bitove resavamo uz pomoć kola. Kritični bitovi su krajnji desni za pomeranje ulevo, kao i krajnji levi za pomeranje udesno. Dakle, samo kontra u odnosu na smer pomeranja, eto kako možemo to lako da zapamtimo. I njih resavamo pomoću kola. Hajde da pogledamo na primer y_0 . Nama je $y_0=0$ za $dir=0$, a za $dir=1$ važi $y_0 = d_1$. Pa to znači da nam y_0 zavisi samo od toga dir , što nam je selekcionni ulaz, a nema veze sa a . Dakle mi trebamo da umesto MUX koji ima izlaz y_0 stavimo dvoulazno I kolo, gde će 1 ulaz da bude selekcionni ulaz iz MUX-a koji ima y_1 za izlaz, a ovaj drugi će biti d_1 . I stavimo y_0 na izlaz. Dakle to je mnogo važno videti u ovom, to šta od čega zavisi. Bas kao i za y_3 , koji je kritičan bit za pomeranje udesno, a nalazi se krajnje levo. Hajde sada za njega da vidimo, za $dir=1$ će biti ili 0 ili d_3 , u zavisnosti od a , pa zato stavljamo I kolo gde će nam jedan ulaz biti a , a drugi d_3 .

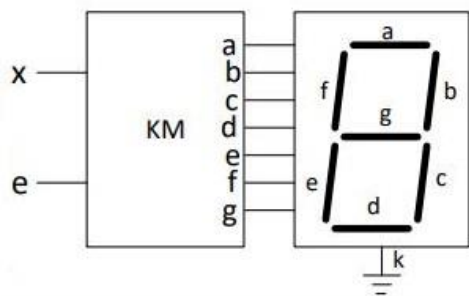
a	dir	y ₃	y ₂	y ₁	y ₀	F-ja
0	0	d ₂	d ₁	d ₀	0	logičko pomeranju ulevo
0	1	0	d ₃	d ₂	d ₁	logičko pomeranje udesno
1	0	d ₂	d ₁	d ₀	0	aritmetičko pomeranje ulevo
1	1	d ₃	d ₃	d ₂	d ₁	aritmetičko pomeranje udesno





*
Како да објаснимо ово: Дакле y_7 је код логичког померања десно нива, а код аритметичког је d_7 . Само да не помешам, значи за померање лево је критичан бит y_0 , али он је нива био да је аритметичко или логичко померање у питању. За померање десно је крајњи леви бит критичан (дакле y_7 ако је 8-битни померај у питању, тј. y_3 за 4-битни). Требао би некако запамтити то да за аритметичко померање десно y_7 има d_7 , а y_3 има d_3 . А за логичко има нива. Свеједно. Много је важно рећи и то да за критичне битове можемо направити оптимизацију I кола. С тим да морам да запамтим да, за померање лево, крајњи десни бит је критичан, тј. y_0 , и он зависи од селекционог улаза dir (који одређује смер померања) и било што тј. MUX 2/1 можемо лењати I колом. Критични бит који се њави кроз лево не зависи од селекционог улаза, те тј. MUX 2/1 не можемо да заменимо I колом. Можемо да ставимо I коло које улази у MUX 2/1.

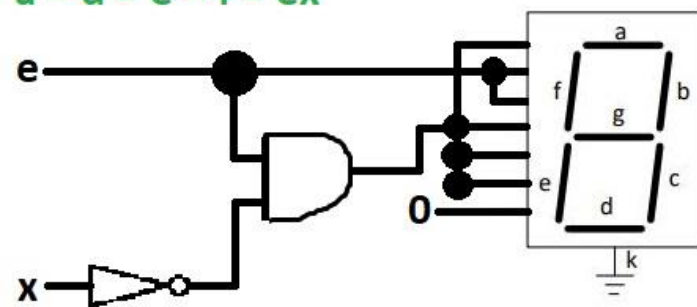
Zadatak: Projektovati kombinacionu mrežu (KM) u sistemu za vizuelnu indikaciju binarne vrednosti (0 ili 1) digitalnog signala (slika). Digitalni signal čija se vrednost prikazuje dovodi se na ulaz x . Rad displeja je omogućen za $e = 1$, dok je za $e = 0$ displej ugašen. Pretpostaviti da segment displeja svetli ako je na njegovom odgovarajućem ulazu 1.



e	x	a	b	c	d	e	f	g
0	X	0	0	0	0	0	0	0
1	0	1	1	1	1	1	0	
1	1	0	1	1	0	0	0	0

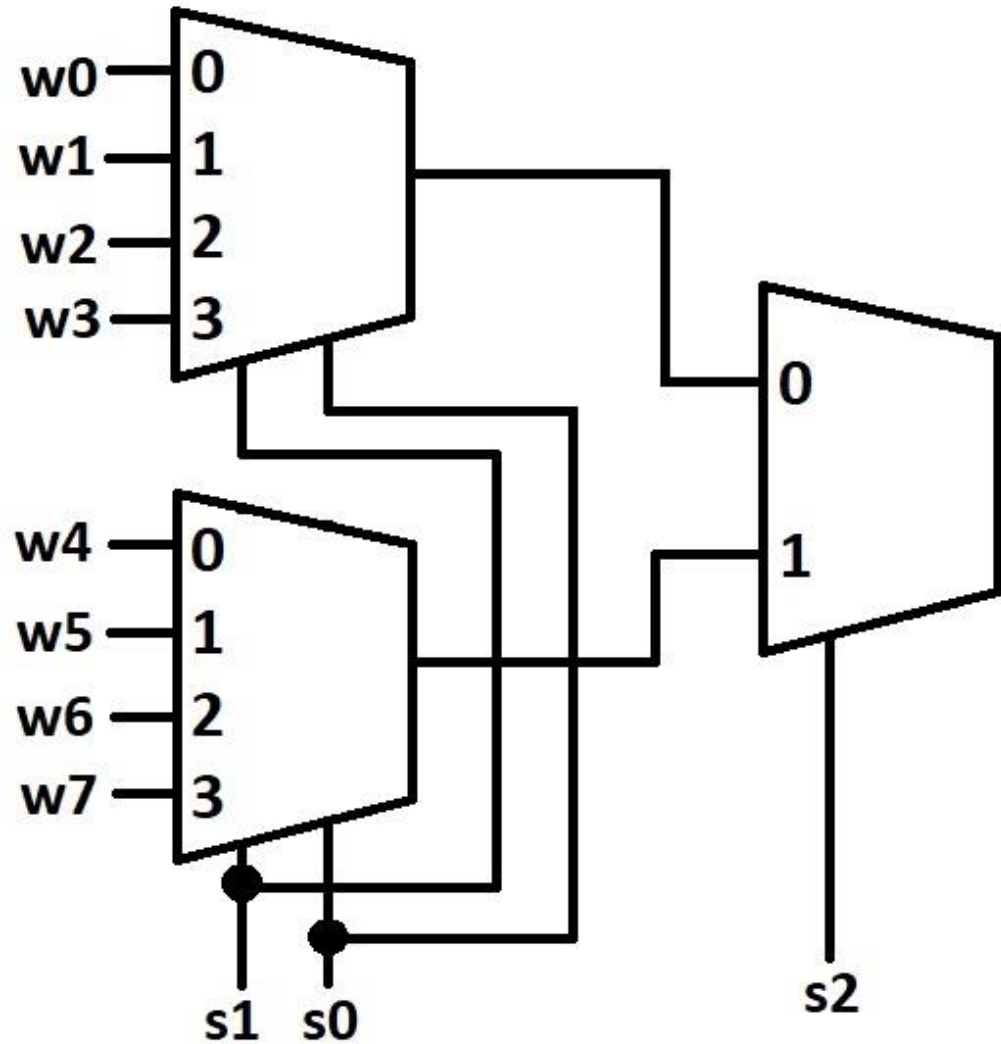
$$b = c = e \quad g = 0$$

$$a = d = e = f = e\bar{x}$$



Realizovati multiplexer 8/1 ako su na raspolaganju:

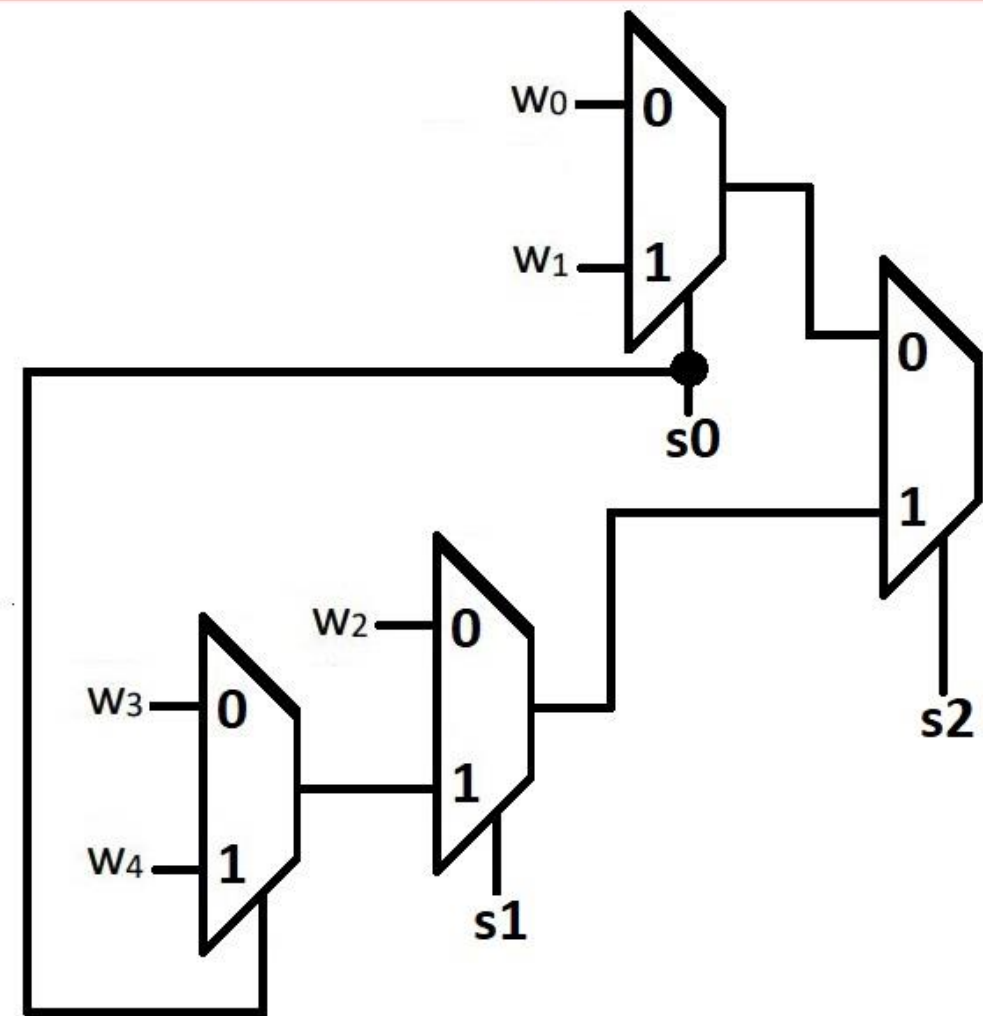
- Dva multipleksera 4/1 i jedan multipleksera 2/1.
- Jedan multiplexer 4/1 i četiri multipleksera 2/1

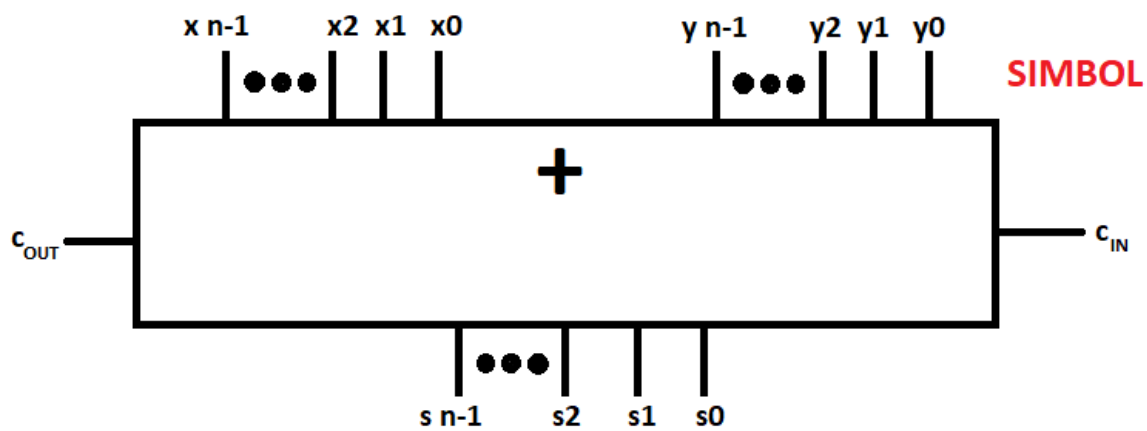
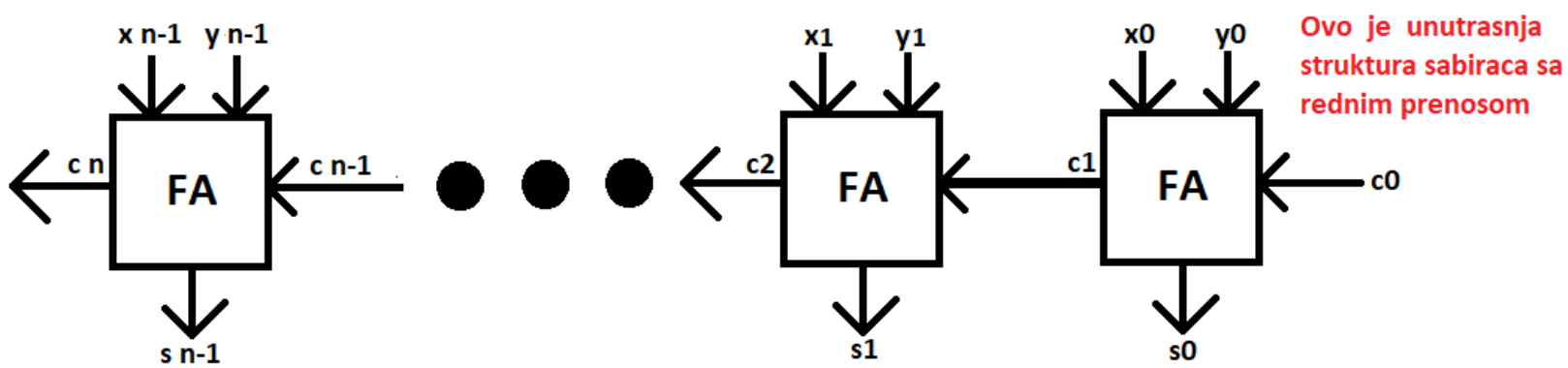


OVDE FALI OBJASNJENJE

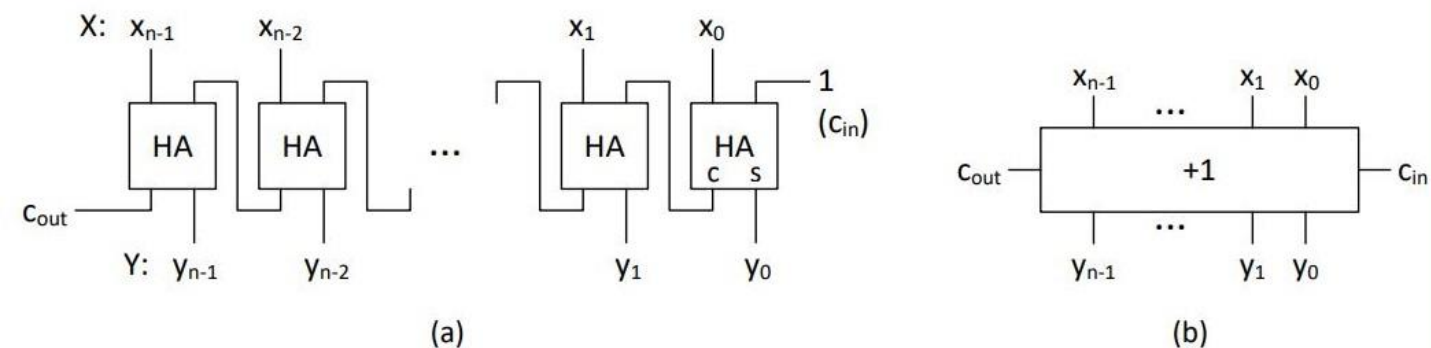
Projektovati multiplekstersku mrežu 5/1, sa ulazima $w_0, \dots, w_4$, izlazom y i tri selekciona signala, s_2, s_1 i s_0 prema funkcionalnoj tabeli na slici. Na raspolaganju su multiplekseri 2/1.

s_2	s_1	s_0	y
0	X	0	w_0
0	X	1	w_1
1	0	X	w_2
1	1	0	w_3
1	1	1	w_4

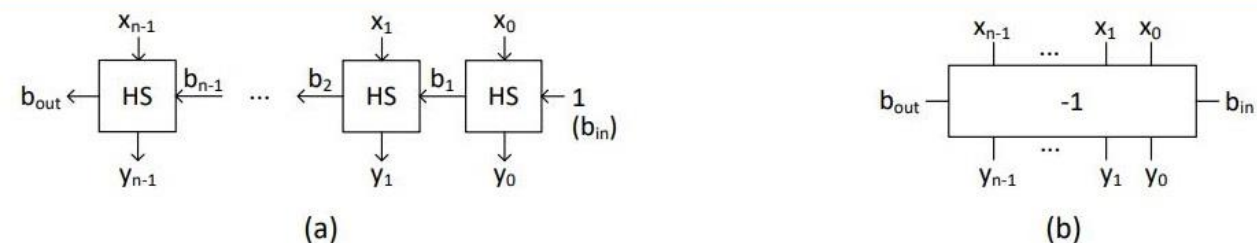




Za sabirac sa rednim prenosom smo ovde dali i strukturu (tj. princip rada) i simbol. Za oduzamac sa rednim prenosom je sve isto bukvalno, samo sto imamo b umesto c za bit prenosa, kao i d umesto s. Sve ostalo je isto bukvalno. I dobro naravno umesto + ide -. I umesto A ide S. Dakle nije FA, vec FS, kao oduzimak.



Sl. 4-9 Realizacija inkrementera pomoću polusabirača: (a) struktura; (b) logički simbol



Sl. 4-12 Realizacija dekrementera pomoću polu-oduzimača: (a) struktura; (b) simbol

Dakle, hajde da objasnimo ovo. Mi za inkrementer koristimo polusabirace, a za dekrementer poluoduzimace. E sad, neke oznake se razlikuju. Na primer, bit prenosa za polusabirac je c , a za poluoduzimac je b . A isto imamo x i y za promenljive. Ajmo ovako sad kad smo to razjasnili. Znaci, za inkrementer, gde su nam polusabiraci koji ga cine, imamo na gornjoj ivici x_0 i 1 , sto nam je c_{in} . Pricamo o polusabiracu koji se nalazi skroz desno. Vazno je reci da nam je to 1 zapravo c_{in} . Dole imamo c i y_0 , na donjoj ivici, opet 2 prikljucka. I ovo c opet povezujemo na pin prenosa sledeceg polusabiraca. Na kraju imamo i c_{out} . Dakle, rezime: x_0 i 1 se sabiraju, imamo dole y_0 i c koji je bit prenosa (na izlazu je tog polusabiraca) i njega povezujemo na ulazni pin prenosa sledeceg polusabiraca. To je rezime. Dakle, imamo: x_i, c, y_i , prenos u sledeci polusabirac. Ovo je uopsten slucaj za i -ti polusabirac. Ukratko objasnjenje kako radi.

Za dekrementer imamo na svakoj stranici kvadrata po 1 pin. Leva i desna strana imaju pin prenosa, koji se ovde zove b_i . Gornja je x_i , a donja strana je y_i . Imamo skroz desno $b_{in} = 1$, a skroz levo b_{out} . Dakle sustinska razlika u konfiguraciji inkrementera i dekrementera je u tome sto se pinovi kod polusabiraca nalaze na gornjoj i donjoj ivici, a kod poluoduzimaca na sve 4 strane.

Открет бројеве које можемо представити у потпуном комплементу са n битова је од $(-1) \cdot (2^{n-1})$ до $2^{n-1} - 1$. Ако хоћемо потпуни комплемент негативног броја некоег, радимо следећи поступак: прво позитивни напишемо бинарно, тако ће нам крајњи леви бит бити 1. Затим додато неку лево од тог крајњег левог бита (важно је да нам број почиње са 01). Потом све битове комплементирамо и тако добијени број саберемо са 1. Пример: ради, најмањи број са моћица 4 бита је 1000, а највећи 0111.

Полузакримач—имамо на улазу x_i и y_i и на излазу b_i и d_i , при чему је b_i пренос а d_i разлика. Имаћемо и случај где је разлика -2 или -1 . Једноставно те бројеве напишемо у потпуном комплементу (гореописани поступак) и то је то. Све ово важи и за потпуни одузимач—улази су x_i, y_i, b_i а излази су b_{i+1} и d_i .

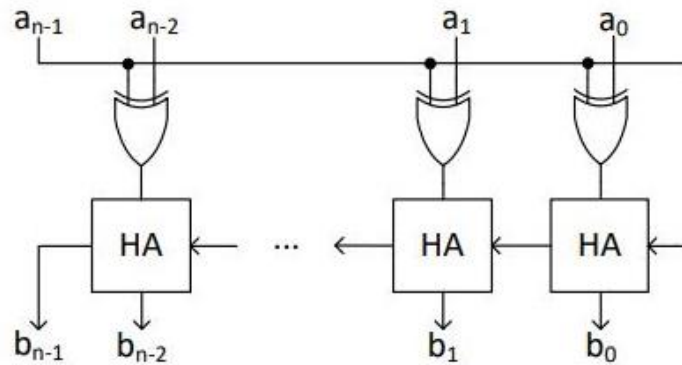
Важно је рећи следећу ствар: када имамо неки негативан број записан у потпуном комплементу, како одређујемо његову апсолутну вредност? Па тако што га цело комплементирамо, и тако добијени број саберемо са 1.

Zadatak 4-5: Projektovati aritmetičko kolo za određivanje apsolutne vrednosti n -bitnog broja u potpunom komplementu.

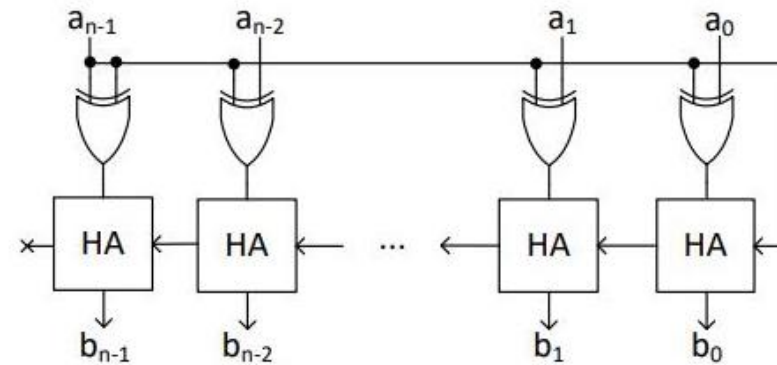
Rešenje: Rešenje je dato na Sl. 4-9(a). Struktura ovog kola direktno sledi na osnovu definicije apsolutne vrednosti:

$$B = |A| = \begin{cases} A, & A \geq 0 \\ -A, & A < 0 \end{cases} = \begin{cases} A, & a_{n-1} = 0 \\ \bar{A} + 1, & a_{n-1} = 1 \end{cases}$$

Ako je A negativan broj, što se vidi po $a_{n-1} = 1$, na komplement broja A se dodaje 1. Efekat ove operacije je promena znaka broja A , s obzirom na identitet $-A = \bar{A} + 1$. Ako je broj A jednak ili veći od nule ($a_{n-1} = 0$), tada broj A prolazi nepromenjen, prvo kroz niz XOR kola, a onda i kroz inkrementer.



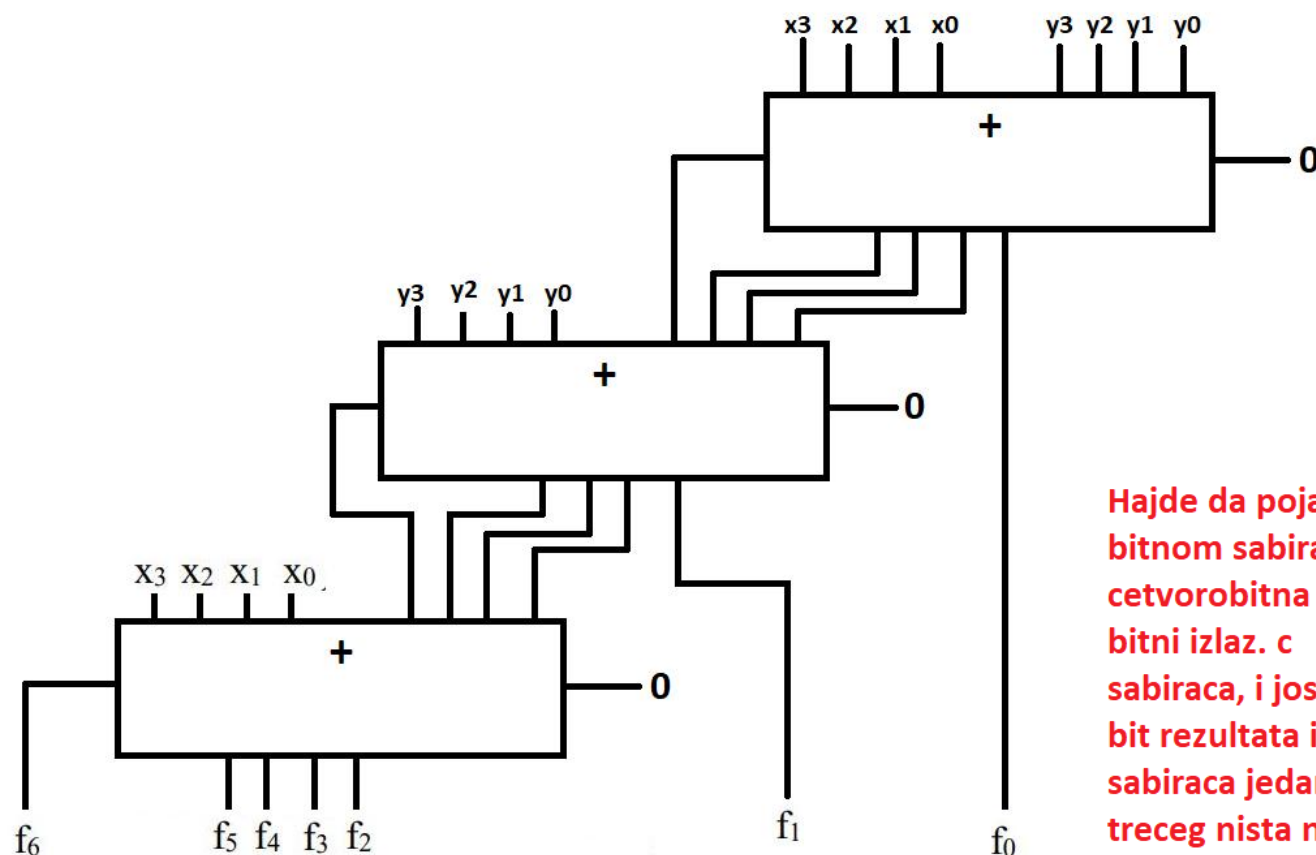
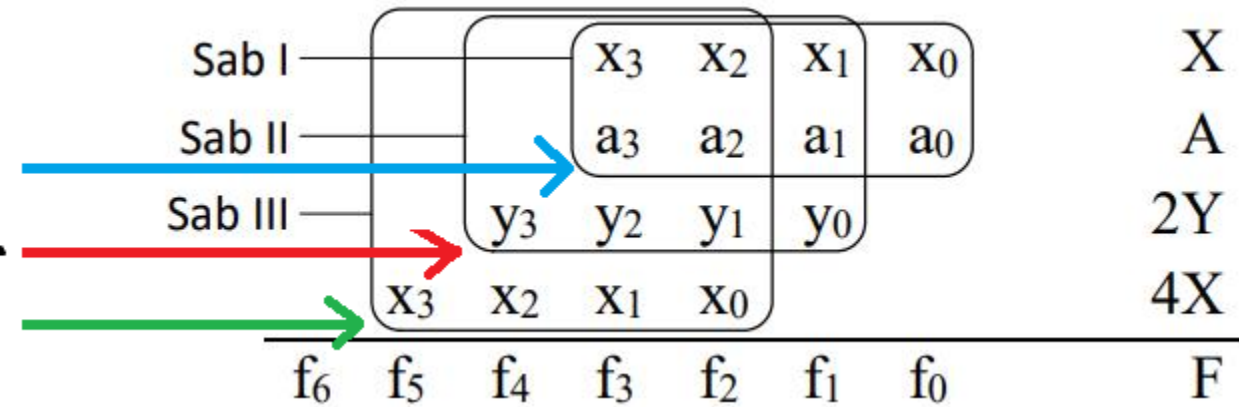
(a)



(b)

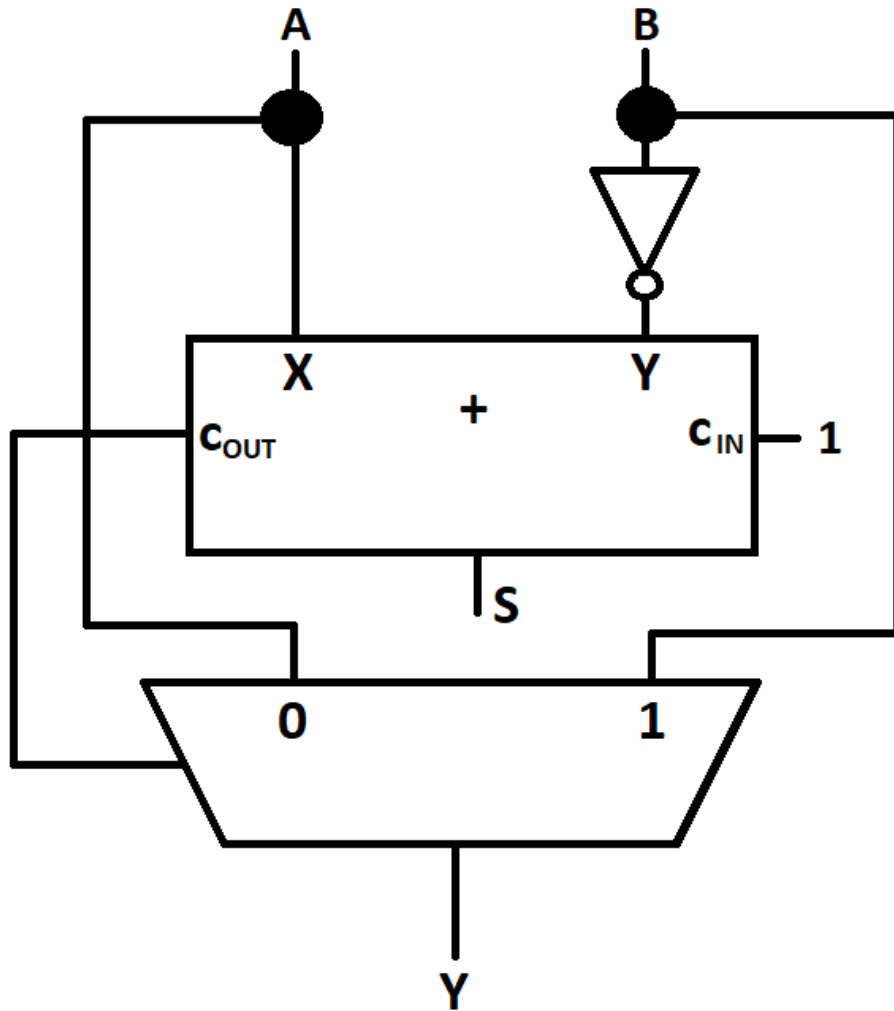
Zadatak: Projektovati aritmetičko kolo za izračunavanje vrednosti funkcije $F = 5X + 2Y + A$, gde su X , Y i A 4-bitni neoznačeni brojevi. Na raspolaganju su 4-bitni sabirači.

Ovo su bitovi najveće
težine i oni moraju biti
obuhvaćeni ovim
okvirom



Hajde da pojasnimo ovo. Znacni, ako pricamo o 4-bitnom sabiracu, mi tu sta imamo. Imamo 2 cetvorobitna broja koja se sabiraju, i imamo 4-bitni izlaz. c ide na ulaz sledeceg 4-bitnog sabiraca, i jos 3 bita rezultata idu na te ulaze, a 1 bit rezultata ide na f_0 i na f_1 . Dakle, od ta prva 2 sabiraca jedan bit ide na f_0 , drugi na f_1 . Od ovog treceg nista ne ide. To je bilo vazno reci.

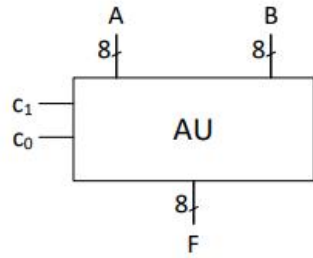
Zadatak: Na bazi binarnog sabirača projektovati aritmetičku jedinicu koja ima dva ulaza za 8-bitne neoznačene binarne brojeve, A i B , i jedan 8-bitni izlaz za neoznačen binarni broj Y , za koji važi $Y = \text{Max}(A, B)$.



Bitno je zapamtiti da nam ide A u obican ulaz multipleksera, bas kao i B , s tim sto nam se B kasnije invertuje. Takodje, treba reci i to da prvo ide sabirac, pa tek onda multiplekser. Zatim, na c_{IN} tog sabiraca ide 1, a c_{OUT} ide na selekcioni ulaz multipleksera. Ovde smo uzeli da nam sabirac ima X i Y ulaze, to su nazivi.

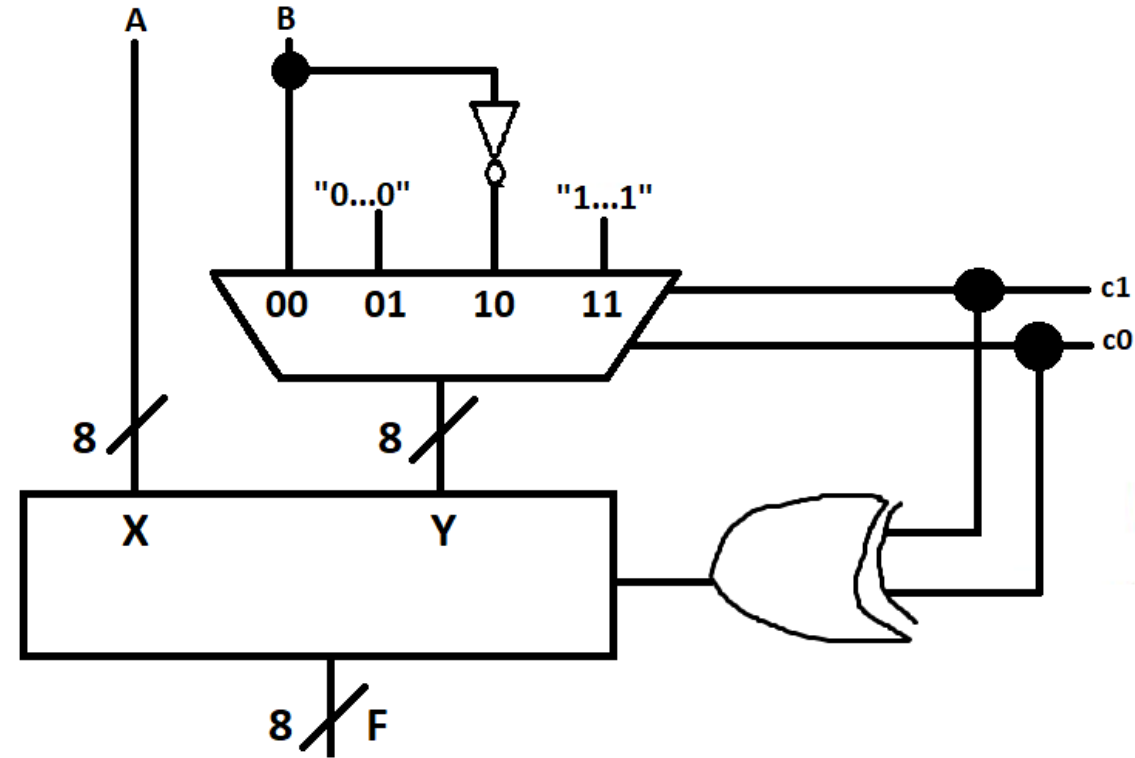
Zadatak: Projektovati aritmetičku jedinicu (AU – *Arithmetic Unit*) prema datoj funkcionalnoj tabeli. AU obavlja jednu od četiri operacije nad dva 8-bitna operanda, A i B , i generiše 8-bitni rezultat, F . Izbor operacije se vrši preko 2-bitnog upravljačkog ulaza (c_1, c_0).

c_1	c_0	Operacija	F
0	0	Sabiranje	$A + B$
0	1	Inkrementiranje	$A + 1$
1	0	Oduzimanje	$A - B$
1	1	Dekrementiranje	$A - 1$



c_1	c_0	Operacija	F	X	Y	C_{IN}
0	0	Sabiranje	$A + B$	A	B	0
0	1	Inkrementiranje	$A + 1$	A	"0...0"	1
1	0	Oduzimanje	$A - B$	A	$\bar{B}$	1
1	1	Dekrementiranje	$A - 1$	A	"1...1"	0

Hajde da pojasnimo sta ce nam ovo XOR kolo. Pa zato sto za kombinaciju 0 i 1 ovih c_1 i c_0 treba da nam vrati kec za ovo c_{in} , a upravo to radi XOR kolo. Sve je logično.



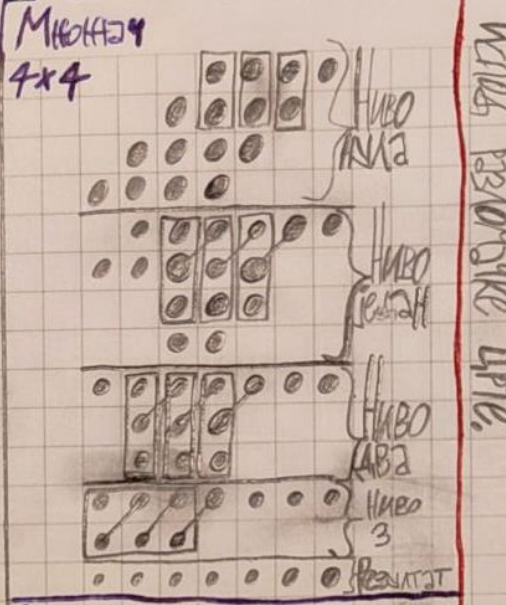
Поента кад се ради ово са овим тачкама је у следећем: ако у колоци
имамо n тачака које су ван правоугаоника, онда ћемо у следећем извоју
опет имати n тачака које немају овакве повезнице  и то у истој
тој колоци. Дале, кад имамо правоугаоник тако, онда у тој истој колоци
имамо повезнице  тако да је прва тачка у тој колоци.
Ум видимо да је правоугаоник, то значи да ће им прва тачка испод црте имати
повезницу. То је важно запамтити.

На нивоу имамо
3 објекта, тако
да на нивоу 1 имамо
три објекта и то
у тој вертикалној
линији. При томе
важни су ове

појаве тачка баш у
тој линији по

вертикали. ~~Востанови~~ ~~оде~~ ~~к~~ ~~мо~~ ~~у~~
Што се тиче објекта у тој вертикали има
непокривених тачака мо тачака и тачака које
има по 1 ниво немају покривених
тачка, толико ће их
бити у нивоу 1.
НАПОМЕНА!!!

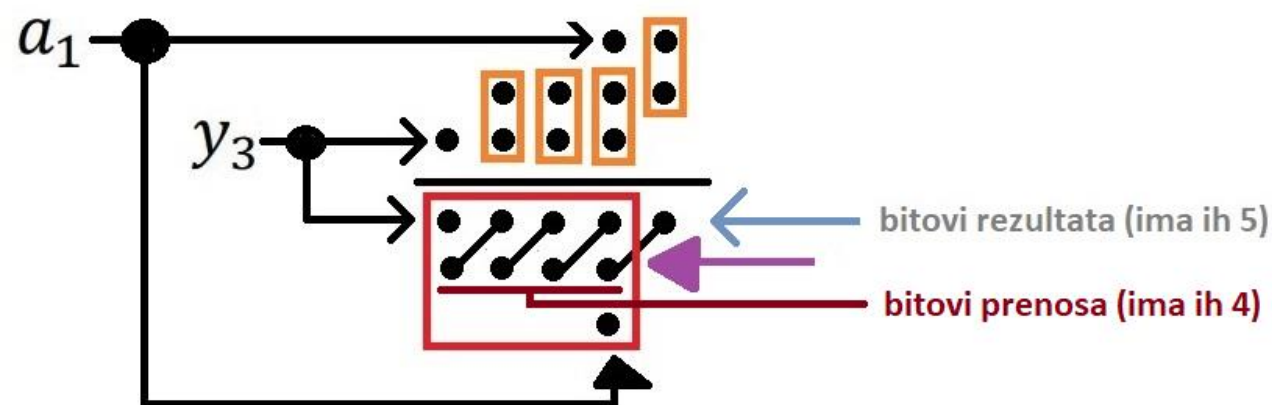
Нема их у овој фо-
рми



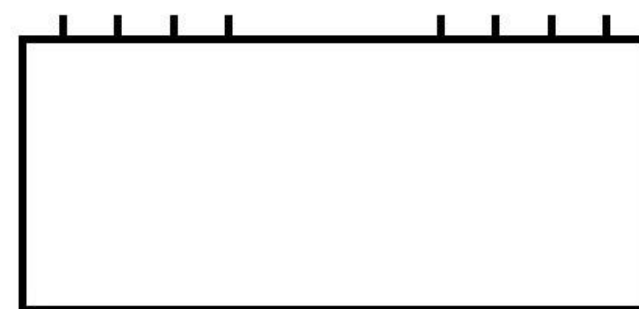
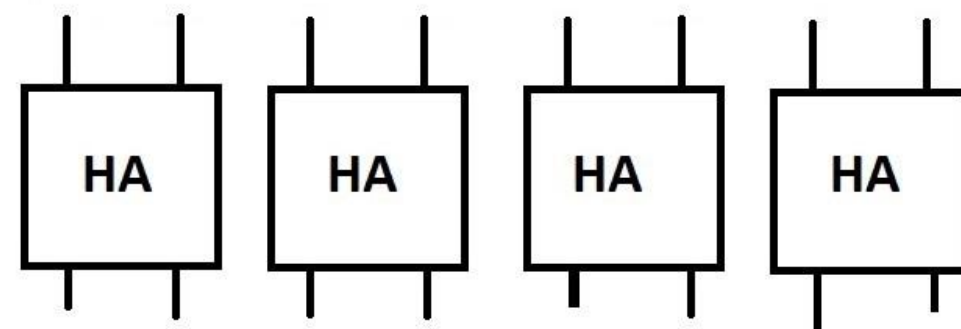
Ako u vertikali na nekom nivou imamo
m tacaka van pra—

u sledecem nivou

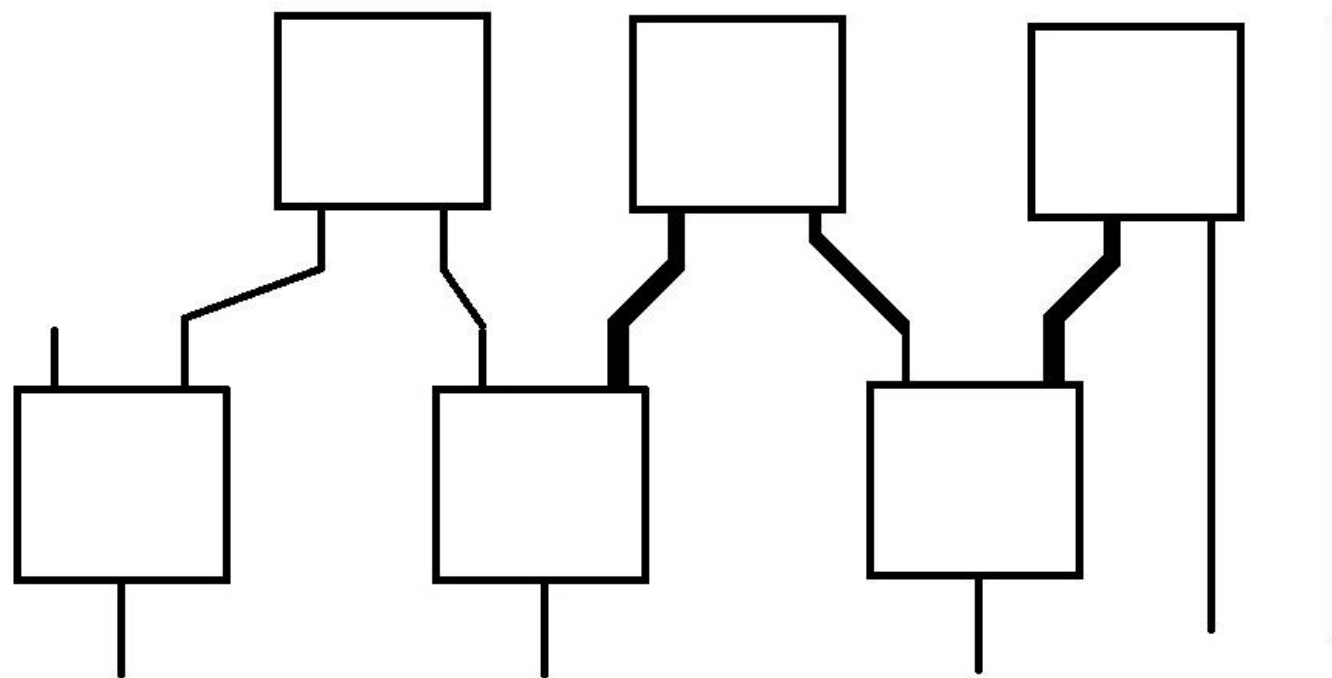
Zadatak: Projektovati aritmetičko kolo za funkciju $F = 3Y + A$, gde je $Y = (y_3y_2y_1y_0)$ 4-bitni i $A = (a_1a_0)$ 2-bitni neoznačeni broj. Na raspolaganju su: jedan 4-bitni sabirač i proizvoljan broj polu-sabirača.



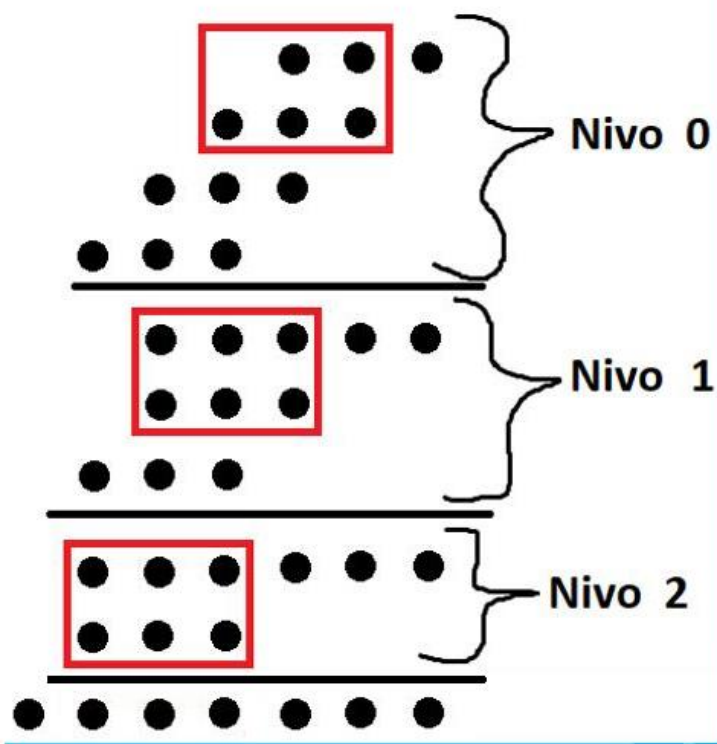
Hajde da objasnimo ovu sliku. Znaci na prethodnim slajdovima je sve objasnjeno sto se tice pravila kako se radi ovo, e sad treba da naglasimo jos 2 vazne stvari. Prva je ta da uvek kod rezultata (crveni pravougaonik) ovaj jedan bit mora da bude van njega, tj. tog pravougaonika (ljubicasta strelica). Druga stvar je da kod ovih poveznica je gornji bit zapravo bit rezultata, a donji bit je bit prenosa. Time se vodimo kad radimo posle realizaciju. Dalje, ajmo sad da vidimo, izdvojili smo bitove y_3 i a_1 . Sta s njima. Pa po pravilu, ovo a_1 ide ispod ovih redova gde su poveznice, i kad je takav slucaj, to stavljamo na c_{IN} . A ovo y_3 jednostavno ubacimo u deo 4-bitnog sabiraca gde su bitovi rezultata. Zato sto je ipak u toj vrsti, a nije povezano ovako. Nema to veze, vazno je da je u toj vrsti. Ovo a_1 nije ni na mestu bita rezultata, ni na mestu bita prenosa, tako da cemo njega da stavimo na c_{IN} .



Zadatak: Projektovati aritmetičko kolo za funkciju $F = 3X + 4Y$, gde je $X = (x_3x_2x_1x_0)$ 4-bitni i $Y = (y_1y_0)$ 2-bitni neoznačeni broj. Na raspolaganju su polu- i potpuni sabirači



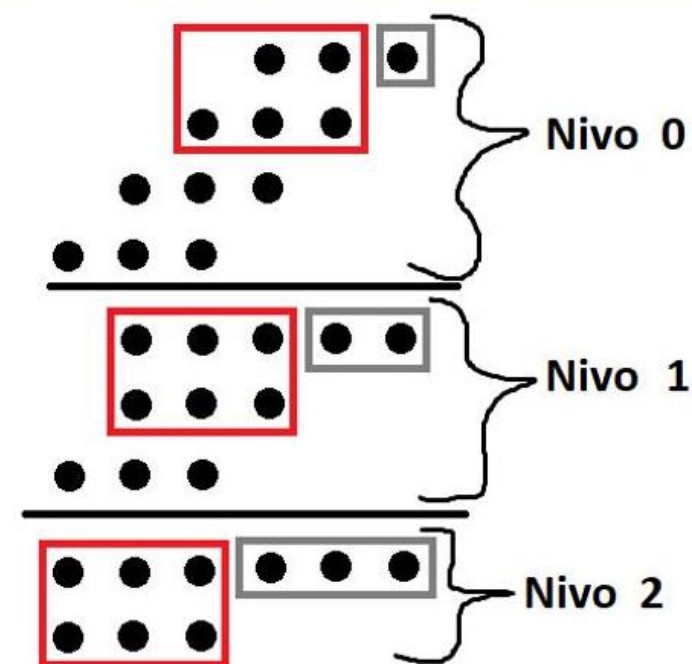
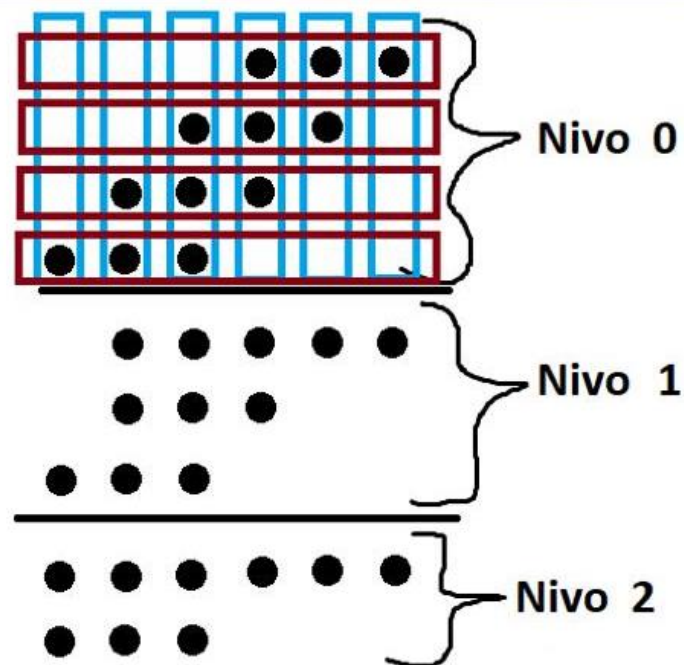
Množać neoznačenih brojeva sa rednim prenosom dimenzije 3 x 4



Hajde da objasnimo ovo: recimo da su nam postavljene tacke vertikalno kolone, a horizontalno vrste. I sad, posto nam treba mnozac 3x4 znaci mi u svakom nivou zaokruzujemo 3 kolone. To mozemo da vidimo na slici. To je prva stvar. Druga stvar je ta da nam na svakom nivou ostaje k bitova koji se nalaze pored pravougaonika. A to k pocinje od 1. U nivou nula je $k = 1$, u nivou 1 je $k = 2$, i u nivou 2 je $k = 3$. I uvek te k-ove moramo da trazimo u prvoj vrsti. Dakle u svakom ovom nivou od nultog do zadnjeg gledamo prvu vrstu za trazenje tih k-ova.

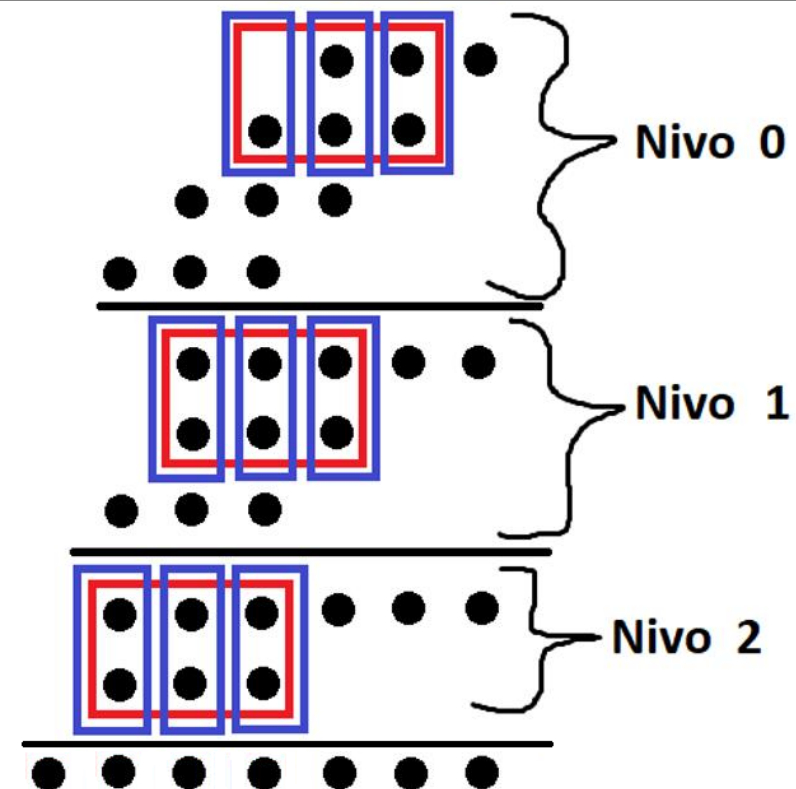
Ok, hajde da objasnimo ovu konstrukciju kako radi. Ovu realizaciju sa crvenim zaokruzenim tackama. Znaci, mi imamo $k = 1$ u nultom nivou. A ta tacka nam je krajnja desna tacka u prvoj vrsti u toj nultoj koloni. Dobro. E pa mi sad zaokruzujemo levo od nje 2 vrste (to uvek radimo, zaokruzujemo 2 vrste kolike god da su dimenzije mnozaca) i zaokruzujemo onoliko kolona kolika je prva brojka u dimenziji mnozaca. Zatim isto odradimo i za nivo 1, ali sada paznja. Koliko smo tacaka zaokruzili u nultom nivou, toliko ih odbacujemo, a ubacujemo tacno 4, jer je to druga brojka u dimenziji mnozaca. To je pravilo. Kolika je druga brojka u dimenziji mnozaca, toliko tacaka ubacujemo u sledeci nivo, a odbacujemo onoliko koliko smo ih zaokruzili. To je univerzalno pravilo za ovakav tip zadatka.

Pomocne slike



Ovo plavo sto je na pomocnoj slici su kolone, a braon jesu vrste. Sto se tice ove druge pomocne slike, tu imamo ovo sivo sto nam predstavlja k. I vidimo da vazi pravilo da je $k = \text{nivo} + 1$. Ovde smo namerno stavili i te k-ove i ovo crveno uokvireno da bi bilo uocljivo kako zaokruzujemo pored njih bitove ove. Vidimo da je u svakom nivou su zaokruzene tacno 3 kolone. I da u nultom nivou imamo $k = 1$, u prvom nivou $k = 2$ itd. Dalje, onoliko tacaka koliko zaokruzimo toliko ih i odbacimo u sledecem nivou, a dodamo onoliko kolika je druga brojka dimenzije mnozaca, u ovom slucaju je to 4. A zaokruzujemo onoliko kolona kolika je prva brojka dimenzije mnozaca, u ovom slucaju je to tri. I eto, te 2 stvari treba da uvek imamo na umu. To je jedna od bitnijih stvari. Sustina.

Množač neoznačenih brojeva sa rednim prenosom dimenzije 3 x 4



Ovi plavi okviri su sabiraci,
tu su uokvireni bitovi koji idu u sabirace.

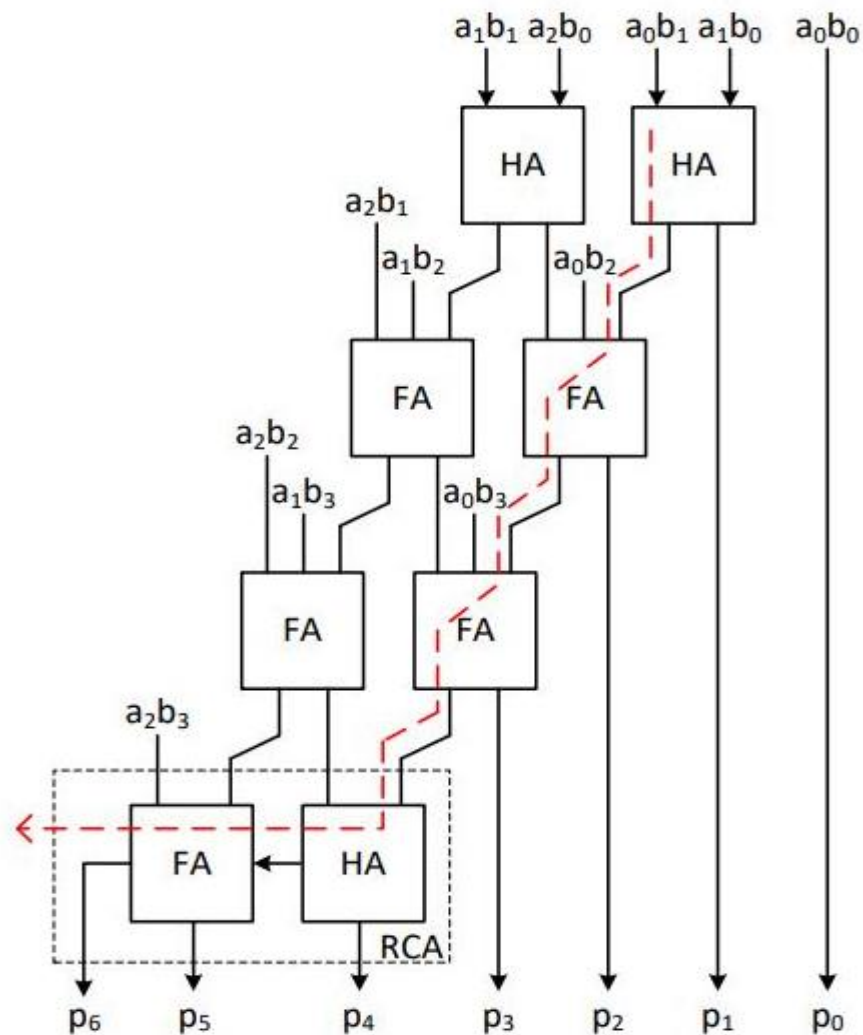
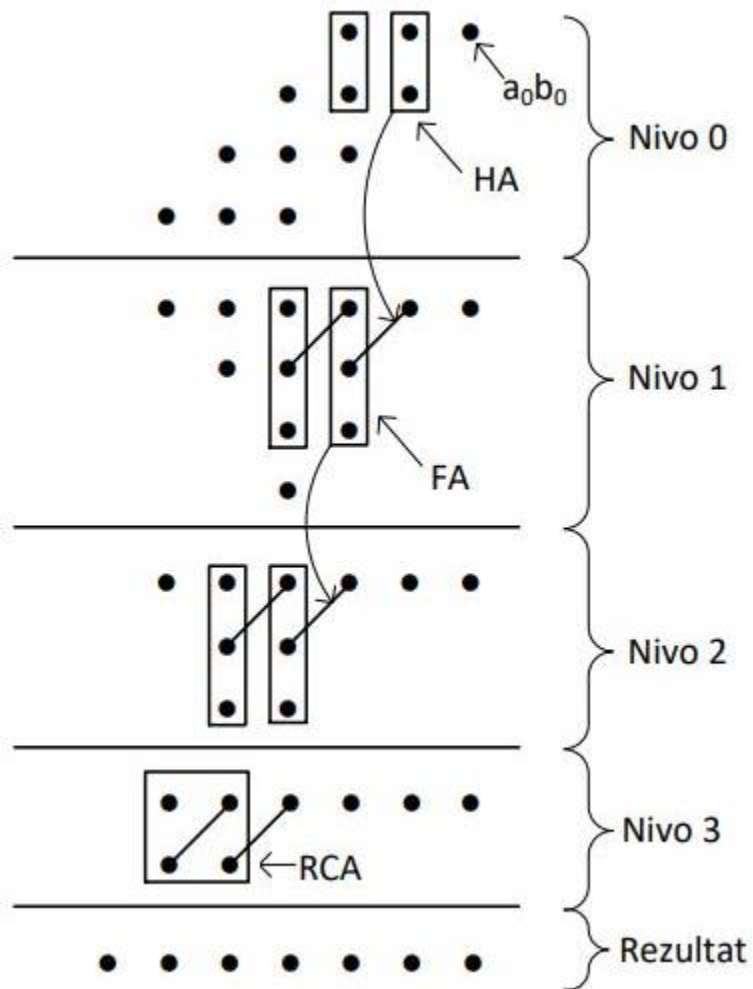
Kako sada realizovati ovo?

				a_2	a_1	a_0	
			b_3	b_2	b_1	b_0	
				a_2b_0	a_1b_0	a_0b_0	I
		a_2b_1	a_1b_1	a_0b_1			II
	a_2b_2	a_1b_2	a_0b_2				III
a_2b_3	a_1b_3	a_0b_3					IV
p_6	p_5	p_4	p_3	p_2	p_1	p_0	

sto bas imamo ovih 7 bitova za rezultat? od p_0 do p_6

b) Matrični množać neoznačenih brojeva.

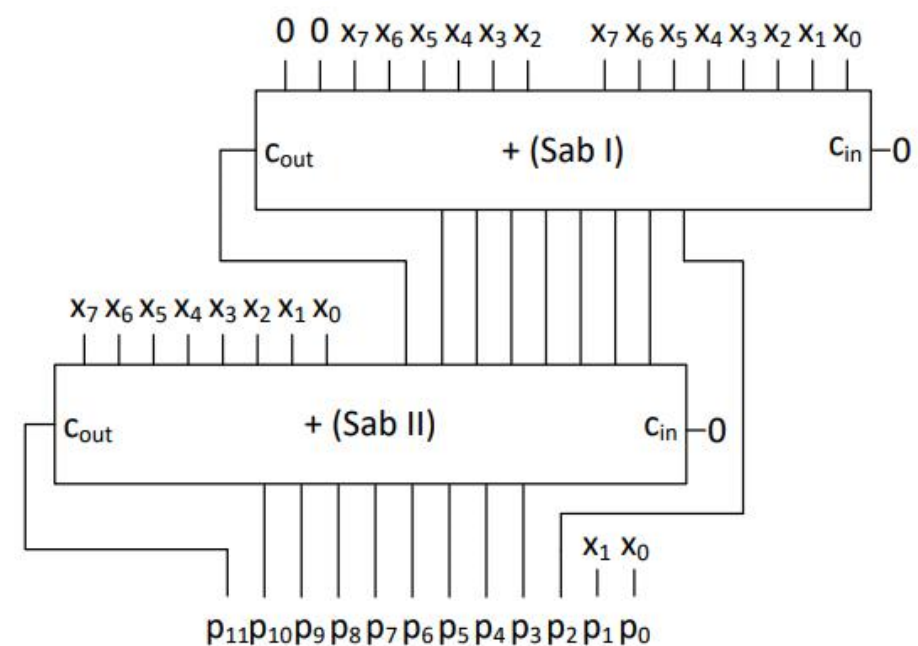
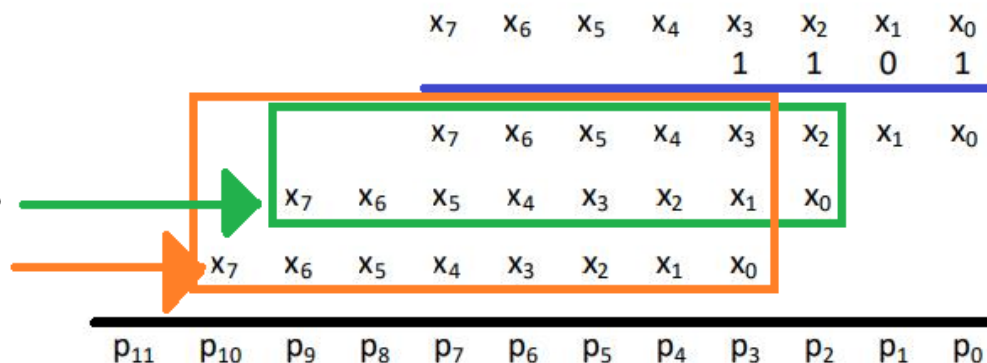
Sta je vazno reci za ovu tackastu metodu. Pa vazno je reci da treba obratiti paznju na ove kolone gde nema pravougaonika, gde nije kolona tacaka uokvirena. Mi i za takvu kolonu razmisljamo na isti nacin kao i za kolonu gde ima pravougaonika koji uokviruje tacke. Dakle koliko je tacaka van pravougaonika, toliko ce ih biti bez poveznice u sledecem nivou. Ako su sve van okvira pravougaonika (tj. pravougaonika ni nema) onda ih sve stavljamo u taj sledeci nivo, po vertikali, i sve su bez one poveznice. Ovo je vazna napomena za sve zadatke gde se ovaj metod radi, ne samo za ovaj zadatak. To je bilo vazno naglasiti. Evo i realizacije.



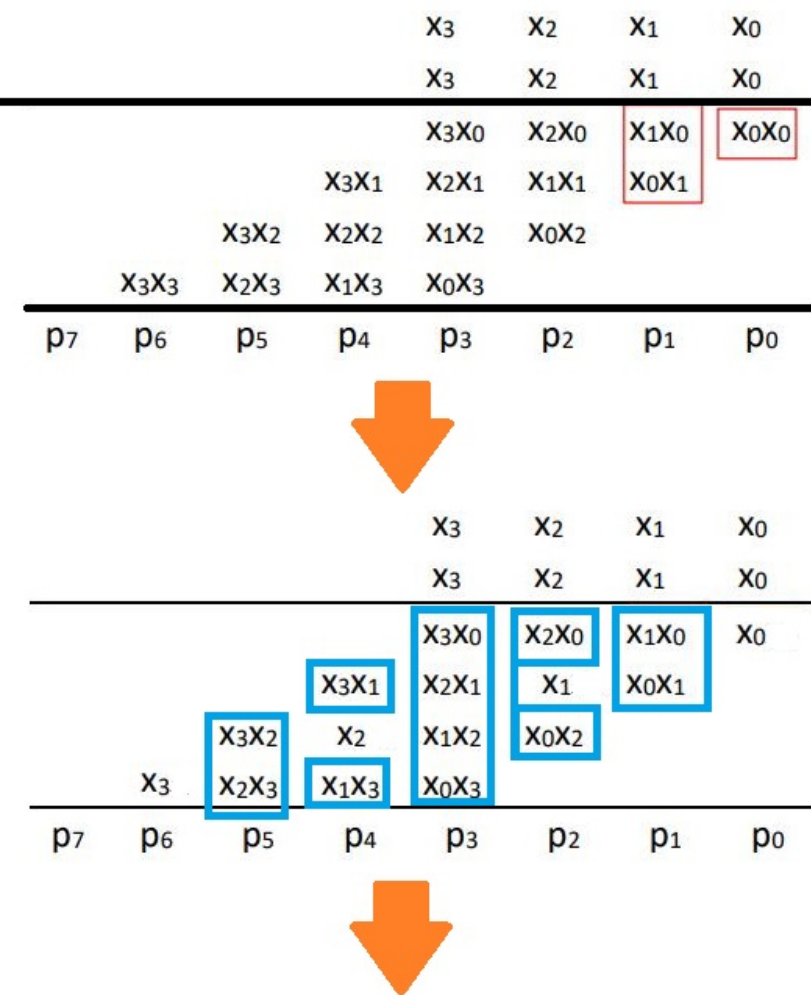
Kako je dobijena ova realizacija?

Projektovati množač 8-bitnog neoznačenog broja X konstantom 13_{10} . Na raspolaganju su binarni sabirači potrebne dužine.

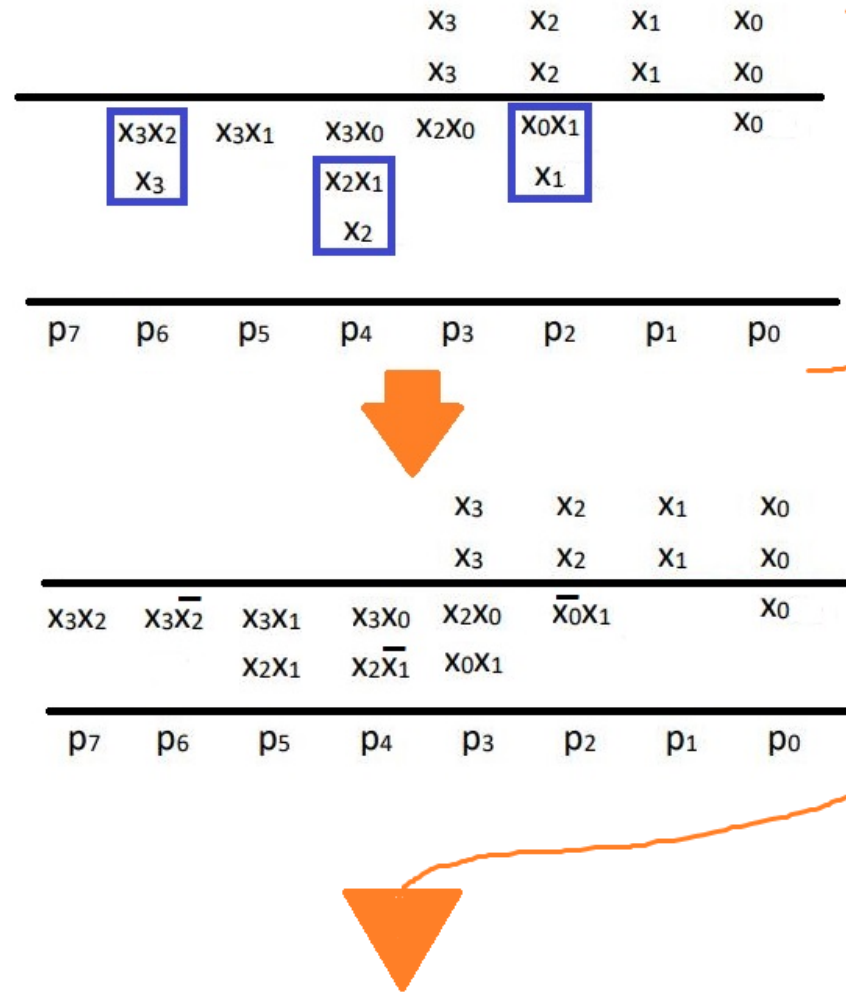
Ovo su bitovi najveće
težine i oni moraju biti
obuhvaćeni ovim
okvirom



Projektovati aritmetičko kolo za kvadriranje 4-bitnog neoznačenog broja.



Objasnjenje za ovo. Ovo plavo sto smo oznacili to su zbirovi koji se prenose u kolonu levo. A ovo sto je ostalo slobodno to su slobodni clanovi. Ti zbirovi kako se prenose to je prikazano na slici gore desno. Vrlo je jednostavno, to su produktni clanovi sa istim ovim bitovima.



Da objasnimo i ovo: Znacii ovo sto je tamnoplavo uokvireno su zbirovi. I sad sta je poenta. Taj red u kojem su tu ostaje ovaj produktni clan sa komplementiranim ovim jednim bitom, dakle bitom koji se sam pojavljuje. A u narednu kolonu ide bez ijednog komplementa. Sve je to prikazano na ovoj zadnjoj slici.

